
KHWOPA ENGINEERING COLLEGE

(Purbanchal University)

DATA SCIENCE

(Elective IV)

Bachelor of Engineering in Computer Engineering

8th Semester



Academic Year: 2082/2083

Khwopa Engineering College

Affiliated to Purbanchal University

Chapter 1

Chapter-1 Introduction to Data Science

What is Data Science?

1.1 Definition and Scope

Data Science is a multidisciplinary field that uses scientific methods, algorithms, processes, and systems to extract knowledge and insights from structured and unstructured data. It sits at the intersection of three core areas:

- **Statistics and Mathematics** – Provides the theoretical foundation for data analysis, probability, distributions, hypothesis testing, and regression.
- **Computer Science and Programming** – Enables data manipulation, algorithm implementation, scalable computing, and automation using languages like Python and R.
- **Domain Expertise** – Allows meaningful interpretation of results within a specific context (finance, healthcare, engineering, etc.).

1.2 Why Data Science Has Become Important

- Explosive growth of data – Every day, 2.5 quintillion bytes of data are created (social media, sensors, transactions, logs).
- Increased computing power – Cloud platforms, GPUs, and distributed computing (Hadoop, Spark) make large-scale analysis feasible.
- Affordable storage – Data storage costs have dropped dramatically.
- Competitive advantage – Organizations use data-driven decisions to outperform competitors.

1.3 Core Activities in Data Science

Activity	Description	Example
Data Collection	Gather data from various sources	Web scraping, APIs, database queries
Data Cleaning	Handle missing values, remove duplicates, fix inconsistencies	Replacing null ages with median
Exploratory Data Analysis (EDA)	Visualize and summarize data characteristics	Histograms, box plots, correlation matrices
Feature Engineering	Create new features from existing data	Extracting day-of-week from timestamp
Modeling	Apply statistical or machine learning algorithms	Linear regression, random forest
Evaluation	Assess model performance	Accuracy, precision, recall, RMSE
Deployment	Put model into production	REST API, batch prediction system
Monitoring	Track model performance over time	Drift detection, retraining triggers

1.0.1 1.4 Types of Data in Data Science

A. By Structure

- Structured – Tabular format (rows and columns). Examples: Excel files, SQL tables, CSV files.
- Semi-structured – Has tags or markers but not rigid tables. Examples: JSON, XML, HTML.
- Unstructured – No predefined format. Examples: text documents, images, videos, audio.

B. By Measurement Scale

- Nominal – Categories without order (colors, yes/no, zip codes).
- Ordinal – Categories with order (ratings: 1st, 2nd, 3rd; education level).
- Interval – Numeric with equal intervals but no true zero (temperature in Celsius).
- Ratio – Numeric with true zero (height, weight, age, income).

C. By Source

- Primary data – Collected directly for a specific purpose (surveys, experiments).
- Secondary data – Already existing (government databases, company logs, public datasets).

1.0.2 1.5 Common Terms in Data Science

Term	Definition
Dataset	A collection of related data records
Observation/Row	A single entity or data point
Feature/Column	An attribute or variable describing the observation
Label/Target	The value to predict (in supervised learning)
Training set	Data used to learn model parameters
Validation set	Data used to tune hyperparameters
Test set	Data used for final evaluation
Overfitting	Model learns noise instead of signal
Underfitting	Model is too simple to capture patterns

Roles of a Data Scientist

2.1 Complete Data Science Team Structure

In a mature organization, multiple roles work together:

Role	Experience Level	Primary Tasks	Typical Tools
Data Analyst	Entry to Mid	Query data, build dashboards, create reports, answer business questions	SQL, Excel, Tableau, Power BI
Data Scientist	Mid to Senior	Build predictive models, experiment design, feature engineering, statistical analysis	Python, R, scikit-learn, TensorFlow, pandas
ML Engineer	Mid to Senior	Deploy models, build pipelines, MLOps, versioning, monitoring	Docker, Kubernetes, MLflow, Airflow
Data Engineer	Mid to Senior	Build data pipelines, ETL processes, data warehousing	Spark, Hadoop, Airflow, AWS Glue
Business Analyst	Mid	Translate business problems into data questions, gather requirements	Jira, Confluence, Excel
Domain Expert	Varies	Provide context, validate results, ensure actionable insights	Domain-specific tools

2.2 Detailed Responsibilities of a Data Scientist

Daily and weekly tasks include:

- **Data Acquisition** – Write SQL queries to extract data from databases; use APIs to fetch real-time data; scrape websites when necessary.
- **Data Cleaning and Wrangling** – Handle missing values (drop or impute); detect and treat outliers; standardize formats; merge multiple data sources.
- **Exploratory Data Analysis (EDA)** – Generate summary statistics (mean, median, standard deviation); create visualizations (scatter plots, box plots, heatmaps); identify patterns, correlations, and anomalies.
- **Feature Engineering** – Create interaction terms; bin continuous variables; encode categorical variables; extract date/time components; create rolling statistics for time series.
- **Model Selection and Training** – Choose appropriate algorithms based on problem type (regression, classification, clustering); split data into train/validation/test sets; train models; tune hyperparameters using cross-validation.
- **Model Evaluation** – Calculate performance metrics (accuracy, precision, recall, F1, ROC-AUC, RMSE, MAE); compare multiple models; perform error analysis.
- **Interpretation and Communication** – Explain model predictions using SHAP or LIME; present findings to non-technical stakeholders; create visual summaries; document methodology.
- **Collaboration** – Work with data engineers to productionize models; consult with domain experts to validate assumptions; mentor junior team members.

2.3 Skills Required for a Data Scientist

Technical Skills (Hard Skills):

- **Programming:** Python (pandas, numpy, scikit-learn, matplotlib, seaborn) or R (tidyverse, caret)
- **SQL:** Joins, subqueries, window functions, aggregations
- **Statistics:** Descriptive statistics, probability, hypothesis testing, regression
- **Machine Learning:** Supervised (regression, classification) and unsupervised (clustering, PCA)
- **Data Visualization:** Matplotlib, seaborn, Plotly, Tableau
- **Big Data Tools:** Spark, Hadoop (for large-scale data)
- **Cloud Platforms:** AWS (S3, SageMaker), GCP (BigQuery), Azure

Soft Skills:

- **Problem-solving** – Breaking down complex business problems
- **Communication** – Explaining technical results to non-technical audiences
- **Curiosity** – Asking the right questions
- **Business acumen** – Understanding what drives value
- **Collaboration** – Working in cross-functional teams

2.4 Typical Data Science Project Effort Distribution

Data Cleaning & Wrangling	60%
Exploratory Data Analysis	20%
Modeling & Validation	15%
Deployment & Communication	5%

This distribution is often cited in industry surveys and reflects why data scientists spend most of their time on data preparation rather than modeling.

Real-World Applications

3.1 Engineering Applications

Predictive Maintenance:

- Sensors collect vibration, temperature, and pressure data from machinery.
- Data science models (anomaly detection, survival analysis, random forests) predict when equipment is likely to fail.
- Maintenance can be scheduled proactively, reducing downtime by 30–50%.

Example: Manufacturing plant with 1000 motors. Historical failure data + real-time sensor readings → model predicts failure probability for each motor. Alerts sent 7 days before predicted failure.

Quality Control:

- Computer vision models inspect products on assembly lines.
- Defects are detected in real-time (scratches, missing components, incorrect dimensions).
- Reduces waste and improves customer satisfaction.

Structural Health Monitoring:

- Strain gauges and accelerometers on bridges, buildings, and dams.
- Time-series analysis detects unusual patterns indicating structural weakness.
- Prevents catastrophic failures.

3.2 Healthcare Applications

Disease Prediction and Diagnosis:

- Electronic Health Records (EHR) contain patient demographics, lab results, medications, and diagnoses.
- Models predict risk of diseases like diabetes, heart disease, or sepsis.
- Early intervention improves outcomes and reduces costs.

Example: Hospital uses logistic regression on patient data (age, BMI, blood pressure, family history) → predicts 5-year heart attack risk with 85% accuracy. High-risk patients receive preventive counseling.

Medical Imaging:

- Deep learning (CNNs) analyzes X-rays, MRIs, CT scans, and pathology slides.
- Detects tumors, fractures, pneumonia, and retinal diseases.
- Can match or exceed radiologist accuracy for specific conditions.

Personalized Medicine:

- Genomic data + treatment outcomes → models predict which drug works best for which patient.
- Reduces trial-and-error prescribing.
- Improves treatment efficacy and reduces side effects.

Hospital Operations:

- Predict patient admission rates to manage staffing.
- Forecast length of stay for bed management.
- Identify readmission risk for follow-up planning.

3.3 Internet of Things (IoT) Applications

Smart Homes:

- Thermostats learn occupancy patterns and adjust temperature automatically.
- Energy consumption is optimized, saving 10–20% on bills.
- Anomaly detection identifies unusual appliance behavior (possible failure or energy waste).

Smart Cities:

- Traffic sensors + cameras + GPS data → real-time traffic flow optimization.
- Adaptive traffic lights reduce congestion by 25%.
- Parking sensors guide drivers to available spots, reducing circling time.

Wearable Devices:

- Fitness trackers and smartwatches collect heart rate, steps, sleep quality, and activity.
- Personalized health insights: “Your recovery score is low. Rest recommended today.”
- Fall detection automatically calls emergency services.

Industrial IoT (IIoT):

- Thousands of sensors on factory floors, oil rigs, and power plants.
- Real-time monitoring prevents accidents and optimizes production.
- Edge computing processes data locally for immediate alerts.

Agriculture (Precision Farming):

- Soil moisture sensors + weather data + crop health imagery → optimize irrigation.
- Reduce water usage by 30% while maintaining yield.
- Drone imagery identifies pest infestations early.

3.4 Finance Applications

Credit Scoring and Risk Assessment:

- Traditional credit scores (FICO) use limited features (payment history, debt).
- Data science models incorporate thousands of features (transaction history, spending patterns, employment stability).
- Better risk prediction reduces default rates and expands credit access.

Fraud Detection:

- Real-time analysis of transactions identifies suspicious patterns.
- Anomaly detection flags unusual purchase locations, amounts, or frequencies.
- Example: Credit card used in two different countries within 1 hour → flagged as potential fraud.

Algorithmic Trading:

- Models analyze market data, news sentiment, and social media.
- High-frequency trading algorithms execute thousands of trades per second.
- Requires careful risk management to avoid flash crashes.

Customer Segmentation and Targeting:

- Clustering algorithms group customers by behavior.
- Personalized product recommendations and interest rate offers.
- Improves customer retention and lifetime value.

Regulatory Compliance (Anti-Money Laundering):

- Network analysis identifies suspicious transaction chains.
- Models flag unusual patterns that may indicate money laundering or terrorist financing.

3.5 Other Important Applications

Retail and E-commerce:

- Recommendation systems ("Customers who bought this also bought...").
- Demand forecasting for inventory management.
- Dynamic pricing based on demand and competitor prices.
- Customer churn prediction with retention offers.

Transportation and Logistics:

- Route optimization for delivery trucks (saving fuel and time).
- Ride-hailing demand prediction (Uber, Lyft).
- Autonomous vehicles (computer vision + sensor fusion + decision making).

Marketing and Advertising:

- Customer lifetime value (CLV) prediction.
- Ad click-through rate (CTR) prediction for ad auctions.
- Sentiment analysis of social media mentions.
- A/B testing for campaign optimization.

Education:

- Student performance prediction for early intervention.
- Personalized learning paths based on mastery.
- Dropout risk identification.

Cybersecurity:

- Network intrusion detection (identifying unusual traffic patterns).
- User behavior analytics (detecting compromised accounts).
- Malware classification using file attributes.

4. Data Science Workflows

4.1 CRISP-DM (Cross-Industry Standard Process for Data Mining)

CRISP-DM was developed in 1996 and remains the most widely used process model for data science projects. It is iterative and non-linear – you can go back to earlier phases as you learn more.

Phase 1: Business Understanding

Goal: Understand project objectives and requirements from a business perspective, then convert this knowledge into a data science problem definition.

Activities:

- Determine business objectives (increase sales? reduce churn? detect fraud?)
- Assess the current situation (available resources, constraints, assumptions)
- Define success criteria in business terms (not just model accuracy)
- Produce a project plan with milestones and deliverables

Example: A bank wants to reduce credit card fraud. Business objective: Reduce fraud losses by 20% within 6 months. Success criteria: Fraud detection rate > 85% with false positive rate < 5%.

Deliverable: Project charter, business success criteria, data science goals (e.g., achieve 0.90 AUC on test set).

Phase 2: Data Understanding

Goal: Collect initial data, describe it, explore it, and verify its quality.

Activities:

- Collect raw data from available sources (databases, files, APIs)
- Load data into analysis environment (pandas DataFrame, R data frame)
- Describe data structure (number of rows, columns, data types)
- Calculate basic statistics (mean, median, min, max, standard deviation for numeric columns; value counts for categorical)
- Explore data visually (histograms, box plots, scatter plots)
- Verify data quality (missing values, duplicates, inconsistencies)

Example (Credit Card Fraud):

- Collect 6 months of transaction data (5 million rows, 30 columns)
- Data types: transaction_id (string), amount (float), timestamp (datetime), merchant_code (string), card_present (boolean), etc.
- Summary statistics: avg amount \$85, median amount \$42, max \$9,999
- Missing values: 2% missing merchant_code, 0.1% missing amount
- Visualizations: Distribution of transaction amounts (highly skewed), fraud rate by hour of day

Deliverable: Data understanding report, initial data quality assessment.

Phase 3: Data Preparation

Goal: Transform raw data into a clean, feature-rich dataset suitable for modeling. This typically consumes 60–80% of project time.

Activities:

A. Data Cleaning:

- Handle missing values:
 - Drop rows with missing target values
 - For numeric features: impute with mean, median, or mode
 - For categorical features: impute with mode or create “missing” category
 - Advanced: KNN imputation or model-based imputation
- Remove duplicates (identical rows or near-duplicates)
- Fix data type errors (string numbers to float, date strings to datetime)
- Detect and treat outliers:
 - Z-score method (values > 3 standard deviations)
 - IQR method (values outside $1.5 \times \text{IQR}$)
 - Domain knowledge (age > 150 is impossible)

B. Data Transformation:

- Scaling/normalization (important for distance-based algorithms):
 - Min-Max scaling: values between 0 and 1
 - Standardization (Z-score): mean 0, standard deviation 1
 - Robust scaling (using median and IQR) – handles outliers better
- Handling skewed distributions: log transformation, Box-Cox transformation
- Encoding categorical variables:
 - Label encoding (assign 0,1,2... for ordinal categories)
 - One-hot encoding (binary columns for each category)
 - Target encoding (replace category with mean of target)

C. Feature Engineering (Creating new features):

- Date/time features: hour of day, day of week, month, day of year, is_weekend
- Aggregations: customer lifetime total spend, average transaction amount, frequency of purchases
- Interaction features: product of two features (e.g., age $\times$ income)
- Ratio features: debt-to-income ratio, body mass index (weight/height²)
- Text features: word count, sentiment score, TF-IDF vectors

D. Data Splitting:

- Training set (60–80%) – used to learn model parameters
- Validation set (10–20%) – used to tune hyperparameters
- Test set (10–20%) – used for final evaluation (untouched until end)

E. Handling Class Imbalance (for classification):

Problem: fraud is rare (0.1% of transactions)

Solutions:

- Undersampling majority class (randomly remove non-fraud)
- Oversampling minority class (duplicate fraud examples)
- SMOTE (Synthetic Minority Oversampling) – creates synthetic examples
- Use class weights in model training

Deliverable: Cleaned, transformed, feature-engineered dataset ready for modeling.

Phase 4: Modeling

Goal: Apply various modeling techniques, tune parameters, and select the best model.

Activities:

A. Algorithm Selection (based on problem type):

Problem Type	Algorithms
Binary Classification (fraud/churn/spam)	Logistic Regression, Decision Tree, Random Forest, XGBoost, SVM, Neural Network
Multi-class Classification (digit/image recognition)	Random Forest, XGBoost, Neural Network, KNN
Regression (price, demand, temperature)	Linear Regression, Ridge/Lasso, Decision Tree, Random Forest, XGBoost
Clustering (segmentation)	K-Means, Hierarchical, DBSCAN
Dimensionality Reduction	PCA, t-SNE, UMAP

B. Training and Validation:

- Train initial baseline model (simple algorithm with default parameters)
- Use cross-validation (typically 5-fold or 10-fold) for stable performance estimates
- Iterate: adjust features, try different algorithms, tune hyperparameters

C. Hyperparameter Tuning:

- Grid Search: exhaustive search over parameter grid
- Random Search: random sampling of parameter space (more efficient)
- Bayesian Optimization: intelligent probabilistic search

Common hyperparameters to tune:

- Decision Tree: max_depth, min_samples_split, min_samples_leaf
- Random Forest: n_estimators, max_features, max_depth
- XGBoost: learning_rate, n_estimators, max_depth, subsample
- Neural Network: number of layers, neurons per layer, learning rate, dropout rate

D. Model Evaluation Metrics:

Problem Type	Metrics
Regression	MAE, RMSE, R-squared (R^2), MAPE
Classification (balanced)	Accuracy, Log Loss
Classification (imbalanced)	Precision, Recall, F1-Score, ROC-AUC, Confusion Matrix
Clustering	Inertia, Silhouette Score, Davies-Bouldin Index

Confusion Matrix (Binary Classification):

Actual \ Predicted	Positive	Negative
Positive	True Positive (TP)	False Negative (FN) – Type II Error
Negative	False Positive (FP) – Type I Error	True Negative (TN)

Precision:

$$\text{Precision} = \frac{TP}{TP + FP}$$

”Of those predicted positive, how many were actually positive?”

Recall:

$$\text{Recall} = \frac{TP}{TP + FN}$$

”Of actual positives, how many were correctly identified?”

F1 Score:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

(Harmonic mean of precision and recall)

ROC-AUC: ROC curve plots True Positive Rate vs False Positive Rate.

- $AUC = 0.5 \rightarrow$ random model
- $AUC = 1.0 \rightarrow$ perfect model
- Good models: $AUC > 0.8$

Deliverable: Best performing model with tuned hyperparameters and evaluated performance metrics.

Phase 5: Evaluation**Goal:** Assess the model against business objectives and determine if it is good enough for deployment.**Activities:**

- Run final model on the unseen test set (one-time evaluation)
- Ensure test performance matches validation performance (check for overfitting)
- Translate technical metrics into business terms:
 - “95% accuracy” → “9,500 out of 10,000 transactions correctly classified”
 - “0.92 ROC-AUC” → “Model ranks fraud cases higher than 92% of non-fraud cases”
- Perform cost-benefit analysis:
 - Cost of false positive (wasted investigation time, customer inconvenience)
 - Cost of false negative (fraud loss, customer liability)
- Find optimal threshold to minimize total cost
- Review against original business success criteria
- Identify limitations or assumptions that could affect production performance

Example (Fraud Detection Business Evaluation):

- Test set: 100,000 transactions (100 fraud, 99,900 non-fraud)
- Model results: 85 fraud correctly detected (Recall 85%), 1,000 flagged as fraud (Precision 8.5%)

Business impact:

- Cost of false positives: $915 \times \$5$ (manual review) = \$4,575
- Savings from detected fraud: $85 \times \$200 = \$17,000$
- Net benefit: \$12,425 per 100,000 transactions

Decision: Proceed to deployment (ROI positive).**Deliverable:** Evaluation report, deployment recommendation, list of remaining risks.**Phase 6: Deployment****Goal:** Put the model into production environment where it can provide business value.**Activities:****A. Deployment Strategies:**

- Batch prediction: Model runs on schedule (nightly/hourly) – used for recommendations, reporting
- Real-time API: Model responds instantly to requests – used for fraud detection, personalization
- Embedded model: Model integrated into application code
- Dashboard: Descriptive analytics using tools like Tableau or Power BI

B. Production Requirements:

- Model serialization (pickle, joblib, ONNX)
- Build prediction service (REST API using Flask/FastAPI)
- Containerization using Docker
- Orchestration using Kubernetes for scaling
- Monitoring:
 - Prediction volume
 - Latency
 - Error rates
 - Data drift
 - Concept drift

C. Documentation:

- Data dictionary (definition of all features)
- Model card (purpose, limitations, performance)
- API documentation (request/response format, endpoints)
- Maintenance plan (retraining schedule, ownership)

D. Retraining Strategy:

- Fixed schedule: monthly or quarterly retraining
- Trigger-based: retrain when performance drops below threshold
- Online learning: incremental updates with new data

Deliverable: Deployed production model, monitoring system, and full documentation.

4.2 OSEMN Model

OSEMN (pronounced "awesome") is a simpler, more data-science-focused workflow. It is ideal for individual projects, Kaggle competitions, and exploratory analysis in notebooks.

Step O: Obtain

Goal: Acquire data from its source into your analysis environment.

Data sources and methods:

Source Type	Method	Example
Local file	read_csv(), read_excel()	pd.read_csv('data.csv')
Database	SQL query via connection	SELECT * FROM transactions
API	HTTP requests (GET, POST)	requests.get('https://api.example.com/data')
Web scraping	BeautifulSoup, Scrapy	Extract table from HTML page
Cloud storage	boto3, google-cloud-storage	Download from S3 bucket
Kaggle	Kaggle API	kaggle competitions download -c titanic

Important considerations:

- Check license and terms of use
- Document data source and retrieval date
- Store raw data unchanged (preserve original)

Output: Raw dataset (may be messy, unprocessed).

Step S: Scrub

Goal: Clean and prepare data for analysis. This is the most time-consuming step.

Detailed scrubbing activities:

1. Initial Assessment:

```
1 df.shape           # (rows, columns)
2 df.info()          # data types and non-null counts
3 df.describe()      # summary statistics
4 df.isnull().sum()  # missing values per column
```

2. Handle Missing Values:

Identify patterns: Are missing values random or systematic?

Numeric columns options:

- Delete rows (if few missing, <5%)
- Impute with mean (symmetric distribution)
- Impute with median (skewed distribution or outliers)
- Impute with mode (most frequent value)
- Forward/backward fill (time series)
- Interpolation (time series with trend)

Categorical columns options:

- Delete rows
- Impute with mode
- Create "Unknown" or "Missing" category
- Advanced: KNN or regression-based imputation

3. Handle Duplicates:

- Exact duplicates: df.drop_duplicates()
- Near duplicates: fuzzy matching or tolerance thresholds

4. Fix Data Types:

```
1 df['amount'] = df['amount'].astype(float)
2 df['date'] = pd.to_datetime(df['date'])
3 df['category'] = df['category'].astype('category')
```

5. Handle Outliers:

Detection methods:

- Z-score > 3 or < -3
- IQR method (outside 1.5 × IQR)
- Domain rules (age > 120, negative price)
- Visualization (box plots, scatter plots)

Treatment options:

- Remove erroneous data
- Winsorization (cap/floor values)
- Log transformation
- Use robust models (tree-based methods)

6. Standardize Formats:

- Text normalization (lowercase, strip whitespace, spelling fixes)
- Date formatting (YYYY-MM-DD)
- Consistent categories (e.g., NY, New York, new york → NY)

Output: Clean, consistent, analysis-ready dataset.

Step E: Explore

Goal: Understand the data through visualization and statistics. Generate hypotheses about patterns and relationships.

Exploratory activities:

1. Univariate Analysis (single variable):

Variable Type	Visualizations	Statistics
Numeric	Histogram, Box plot, Density plot	Mean, Median, Mode, Std Dev, Skewness, Kurtosis, Min, Max, Quartiles
Categorical	Bar chart, Pie chart	Frequency count, Percentage, Mode

Questions to answer:

- What is the central tendency of the variable?
- How spread out are the values?
- Is the distribution symmetric or skewed?
- Are there any clusters or gaps?

2. Bivariate Analysis (two variables):

Combination	Visualizations	Statistics
Numeric vs Numeric	Scatter plot, Hexbin plot, Line plot	Pearson correlation, Spearman correlation
Numeric vs Categorical	Box plot, Violin plot, Bar chart (means)	ANOVA, t-test, Grouped stats
Categorical vs Categorical	Stacked bar chart, Mosaic plot	Chi-square test, Cramér's V

Questions to answer:

- Are variables correlated (positive, negative, none)?
- Is relationship linear or non-linear?
- Does category affect numeric variable?
- Are variables independent?

3. Multivariate Analysis (3+ variables):

- Correlation matrix (heatmap)
- Pairplots (scatter matrix)
- Parallel coordinates
- 3D scatter plots (limited usefulness)
- Facet grids / small multiples

4. Missing Data Patterns:

- Visualize missingness (e.g., missingno library)
- Check correlation of missing values across columns

5. Hypothesis Generation:

- Higher income customers have higher purchase amounts
- Fraud is more common for weekend transactions
- Churn risk increases after 3 months of inactivity

Output: Visualizations, statistical summaries, and hypotheses for modeling.

Step M: Model

Goal: Apply statistical or machine learning models to make predictions, find patterns, or test hypotheses.

Modeling approaches by goal:

Goal	Approach	Example Algorithms
Prediction (continuous)	Regression	Linear, Ridge, Lasso, Random Forest, XGBoost
Prediction (category)	Classification	Logistic Regression, SVM, Random Forest, Neural Net
Pattern discovery	Clustering	K-Means, Hierarchical, DBSCAN
Dimensionality reduction	Feature extraction	PCA, t-SNE
Relationship testing	Statistical inference	Hypothesis tests, Confidence intervals

Basic modeling workflow:

1. Prepare features and target

- X = input features
- y = target variable

2. Train-test split

```
1 from sklearn.model_selection import train_test_split
2
3 X_train, X_test, y_train, y_test = train_test_split(
4     X, y, test_size=0.2, random_state=42
5 )
```

3. Train model

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 model = RandomForestClassifier()
4 model.fit(X_train, y_train)
```

4. Predict

```
1 y_pred = model.predict(X_test)
2 y_pred_proba = model.predict_proba(X_test)[:, 1]
```

5. Evaluate

```
1 from sklearn.metrics import accuracy_score, roc_auc_score
2
3 print(f"Accuracy: {accuracy_score(y_test, y_pred)}")
4 print(f"AUC: {roc_auc_score(y_test, y_pred_proba)}")
```

Output: Trained model with performance metrics.

Step I: Interpret

Goal: Translate model outputs into actionable insights for stakeholders.

Interpretation activities:

1. Model Explanation (non-technical audience):

- What does the model do? (one simple sentence)
- How well does it perform? (simple metrics, avoid jargon)
- What features are most important? (top 5 feature chart)
- When does the model fail? (limitations and edge cases)

2. Feature Importance:

- Identify most influential features
- Understand direction of impact (positive/negative)

3. Individual Prediction Explanation:

- SHAP (SHapley Additive exPlanations)
- LIME (Local Interpretable Model-agnostic Explanations)

Example explanation:

”This transaction was flagged as fraud because: amount is unusually high (\$950 vs typical \$45), location is new (first time in country X), and time is unusual (3 AM).”

4. Business Recommendations:

- Translate findings into actions
- Example: Customers inactive for 30 days have 60% churn risk. Send retention offer on day 25.

5. Visual Summary:

- Confusion matrix visualization
- ROC curve
- Feature importance bar chart
- Example predictions with explanations

Output: Clear report or presentation with actionable recommendations.

4.3 CRISP-DM vs OSEMN: Detailed Comparison

Aspect	CRISP-DM	OSEMN
Number of phases	6	5
Focus	Full project lifecycle	Analytical workflow
Business focus	Strong (Phase 1 explicitly business)	Weak (starts at obtaining data)
Deployment included	Yes (Phase 6)	No (stops at interpretation)
Evaluation	Separate phase	Part of interpretation
Iteration	Explicitly iterative	Implied iteration
Best for	Enterprise team projects	Individual/Kaggle/notebook work
Documentation	Heavy (requires project documentation)	Light
Time perspective	Weeks to months	Hours to days
Origin	Industry consortium (1996)	Data science community (2000s)
Learning curve	Steeper	Gentler

When to use CRISP-DM:

- Working in a team on a production project
- Need formal approval at each stage
- Business stakeholders require regular updates
- Long-term project with maintenance phase

When to use OSEMN:

- Exploring a new dataset independently
- Kaggle competition or personal project
- Quick proof-of-concept
- Teaching or learning data science

Practice Questions

SECTION A: 2-Mark Questions

1. Define Data Science.
2. What are the three core components of Data Science?
3. List the six phases of the CRISP-DM model.
4. List the five steps of the OSEMN model.
5. What is the difference between structured and unstructured data? Give one example of each.
6. Define overfitting and underfitting.
7. What is the typical percentage of time a Data Scientist spends on data cleaning?
8. What is Exploratory Data Analysis (EDA)?
9. What is the difference between a training set and a test set?
10. Define precision and recall.

SECTION B: 4-Mark Questions

1. Explain the three core components of Data Science with suitable examples.
2. Describe any four real-world applications of Data Science in healthcare.
3. Describe any four real-world applications of Data Science in IoT.
4. Describe any four real-world applications of Data Science in finance.
5. Differentiate between a Data Analyst, Data Scientist, and ML Engineer (any four points).
6. Compare CRISP-DM and OSEMN models on any four parameters.
7. Explain the difference between overfitting and underfitting. How can each be detected and prevented?
8. Explain any four activities performed during the Data Preparation phase of CRISP-DM.
9. What is feature engineering? Explain with two examples.
10. Explain the confusion matrix for binary classification with a diagram. Define TP, FP, TN, FN.

SECTION C: 8-Mark Questions

1. Explain the CRISP-DM methodology in detail. Discuss all six phases with suitable examples from a real-world data science project.
2. Explain the OSEMN model in detail. Discuss all five steps with suitable examples from a real-world data science project.
3. Compare and contrast the CRISP-DM and OSEMN workflows in detail. Under what circumstances would you prefer one over the other?
4. Discuss the various roles in a data science team (Data Analyst, Data Scientist, ML Engineer, Data Engineer, Domain Expert). Explain their responsibilities, required skills, and how they collaborate.
5. With suitable examples from at least three different domains (healthcare, finance, IoT, engineering, retail), explain how data science is transforming real-world decision-making.
6. Explain the complete modeling workflow in data science including: algorithm selection, train-validation-test split, training, hyperparameter tuning, and evaluation metrics (accuracy, precision, recall, F1, ROC-AUC).
7. Explain the Data Preparation phase of CRISP-DM in detail. Include data cleaning (missing values, outliers, duplicates), data transformation (scaling, encoding), and feature engineering with examples.
8. Explain the Evaluation and Deployment phases of CRISP-DM in detail. Include evaluation metrics, cost-benefit analysis, deployment strategies (batch, real-time API), monitoring, and retraining.
9. You are a Data Scientist working on a credit card fraud detection project. Walk through the entire CRISP-DM workflow from Business Understanding to Deployment with specific examples at each phase.
10. Compare and contrast the four types of data by measurement scale: nominal, ordinal, interval, and ratio. Give two examples of each and explain why the distinction matters in data science.

Chapter 2

Chapter-2 Data Collection and Preprocessing

What is Data

Data refers to raw facts, figures, or observations that are collected for analysis or reference.

It can exist in many forms such as numbers, text, images, audio, video, or symbols. On its own, data has no meaning until it is processed and interpreted.

For example, the numbers 85, 90, 78 are data. When organized as “student marks in a test,” they become meaningful information.

In simple terms:

- Data = raw input
- Information = processed, meaningful data
- Knowledge = insights derived from information

Data Types (Classification by Structure)

Data Type	Description	Examples	Advantages / Challenges
Structured Data	Organized in predefined rows and columns with fixed schema	CSV files, Excel spreadsheets, SQL tables	Easy to query, store, and analyze; efficient storage / Limited flexibility; schema changes are difficult
Semi-structured Data	Has tags/markers to separate elements but no rigid schema	JSON, XML, HTML, YAML	Flexible schema; self-describing; hierarchical representation / More complex to parse than structured; increased storage overhead
Unstructured Data	No predefined format or organization	Text documents, images, videos, audio, social media posts	Rich information content; human-readable formats / Difficult to analyze; requires NLP/CV techniques; large storage needed

Data Types (Classification by Measurement Scale)

Scale Type	Description	Mathematical Operations	Examples
Nominal	Categories with no order or ranking	Mode, frequency count	Gender (Male/Female), colors, yes/no, zip codes
Ordinal	Categories with meaningful order but unequal intervals	Mode, median, percentile	Education (High School < Bachelor < Master < PhD), ratings (1–5 stars)
Interval	Numeric with equal intervals but no true zero	+, -, mean, standard deviation	Temperature (Celsius/Fahrenheit), dates, IQ scores
Ratio	Numeric with equal intervals AND a true zero	+, -, ×, $\nabla\bullet$, all statistical operations	Height, weight, age, income, price, counts

Data Sources and Collection Methods

Source Type	Method	Use Case	Example
Local Files	<code>pd.read_csv()</code> , <code>pd.read_excel()</code> , <code>pd.read_json()</code>	Small to medium datasets on your computer	<code>sales_data.csv</code> , <code>customer.xlsx</code>
Databases	SQL queries (SELECT, JOIN, WHERE)	Large structured data requiring querying	PostgreSQL, MySQL, Oracle, SQL Server
APIs (REST)	HTTP requests (GET, POST, PUT, DELETE)	Real-time or third-party data	Twitter API, Weather API, Google Maps API
Web Scraping	BeautifulSoup, Selenium, Scrapy	Data not available through APIs	Extracting product prices from e-commerce sites
Cloud Storage	boto3 (AWS S3), google-cloud-storage	Scalable storage of large datasets	Data lakes, backup storage
IoT Sensors	MQTT, CoAP protocols	Real-time streaming data	Temperature sensors, fitness trackers

Working with Different Data Types in Data Science

1. What is a CSV File?

CSV (Comma-Separated Values) is a simple file format used to store tabular data, such as a spreadsheet or database table.

1.1 Structure of a CSV File

- Each line in the file represents a row of data
- Values within a row are separated by delimiters (usually commas)
- The first row often contains column headers

1.2 Example of a CSV File

File: `customers.csv` (raw format)

```
1 customer_id,name,age,city,amount
2 1,Alice,25,New York,250.50
3 2,Bob,30,Los Angeles,180.00
4 3,Charlie,35,Chicago,320.75
5 4,Diana,28,Houston,95.25
6 5,Eve,42,Phoenix,510.00
```

Same data displayed as a table:

customer_id	name	age	city	amount
1	Alice	25	New York	250.50
2	Bob	30	Los Angeles	180.00
3	Charlie	35	Chicago	320.75
4	Diana	28	Houston	95.25
5	Eve	42	Phoenix	510.00

1.3 Features of CSV Files

Feature	Description
Human-readable	Can be opened in any text editor
Lightweight	Smaller file size compared to Excel
Universal compatibility	Supported by Excel, Google Sheets, databases, programming languages
No formatting	Stores only data, not formulas or styling
Simple structure	Easy to parse and generate

1.4 CSV Delimiters (Alternatives to Comma)

Delimiter	File Extension	Region / Use
Comma (,)	.csv	Most common, English regions
Semicolon (;)	.csv	European regions (Excel uses semicolon)
Tab (\t)	.tsv or .txt	When data contains commas
Pipe (—)	.csv	When data contains commas and tabs
Space ()	.txt	Simple data without spaces

2. Method 1: Using Python’s Built-in CSV Module

The `csv` module is Python’s built-in library for reading and writing CSV files. It does not require any installation.

2.1 Importing the CSV Module

```
1 import csv
```

2.2 CSV Reader – Reading CSV Files

2.2.1 Basic CSV Reader

```
1 import csv
2
3 # Open and read CSV file
4 with open('customers.csv', 'r') as file:
5     csv_reader = csv.reader(file)
6
7     # Read header row
8     header = next(csv_reader)
9     print(f"Headers: {header}")
10
11    # Read all data rows
12    print("\nData rows:")
13    for row in csv_reader:
14        print(row)
```

Output:

```
1 Headers: ['customer_id', 'name', 'age', 'city', 'amount']
2
3 Data rows:
4 ['1', 'Alice', '25', 'New York', '250.50']
5 ['2', 'Bob', '30', 'Los Angeles', '180.00']
6 ['3', 'Charlie', '35', 'Chicago', '320.75']
7 ['4', 'Diana', '28', 'Houston', '95.25']
8 ['5', 'Eve', '42', 'Phoenix', '510.00']
```

2.2.2 CSV Reader with Different Delimiter

```
1 import csv
2
3 # Reading a semicolon-separated file
4 with open('customers_semicolon.csv', 'r') as file:
5     csv_reader = csv.reader(file, delimiter=';')
6
7     for row in csv_reader:
8         print(row)
```

2.2.3 CSV DictReader (Read as Dictionaries)

```
1 import csv
2
3 with open('customers.csv', 'r') as file:
4     csv_reader = csv.DictReader(file)
5
6     for row in csv_reader:
7         print(f"Name: {row['name']}, City: {row['city']}, Amount: {row['amount']}")
```

Output:

```
1 Name: Alice, City: New York, Amount: 250.50
2 Name: Bob, City: Los Angeles, Amount: 180.00
3 Name: Charlie, City: Chicago, Amount: 320.75
4 Name: Diana, City: Houston, Amount: 95.25
5 Name: Eve, City: Phoenix, Amount: 510.00
```

2.2.4 Reading CSV into a List

```
1 import csv
2
3 with open('customers.csv', 'r') as file:
4     csv_reader = csv.reader(file)
5     data = list(csv_reader)
6
7 print(f"Total rows: {len(data)}")
8 print(f"First row (headers): {data[0]}")
9 print(f"Second row (first data row): {data[1]}")
```

2.2.5 Reading Specific Columns

```
1 import csv
2
3 with open('customers.csv', 'r') as file:
4     csv_reader = csv.DictReader(file)
5
6     for row in csv_reader:
7         name = row['name']
8         amount = row['amount']
9         print(f"{name}: ${amount}")
```

2.3 CSV Writer – Writing CSV Files

2.3.1 Basic CSV Writer

```
1 import csv
2
3 # Data to write
```

```

4 data = [
5     ['customer_id', 'name', 'age', 'city', 'amount'],
6     [1, 'Alice', 25, 'New York', 250.50],
7     [2, 'Bob', 30, 'Los Angeles', 180.00],
8     [3, 'Charlie', 35, 'Chicago', 320.75]
9 ]
10
11 # Write to CSV file
12 with open('output_csv_writer.csv', 'w', newline='') as file:
13     csv_writer = csv.writer(file)
14
15     csv_writer.writerows(data)
16
17 print("CSV file created successfully!")

```

2.3.2 Writing Row by Row

```

1 import csv
2
3 with open('output_row_by_row.csv', 'w', newline='') as file:
4     csv_writer = csv.writer(file)
5
6     csv_writer.writerow(['customer_id', 'name', 'age', 'city', 'amount'])
7
8     csv_writer.writerow([1, 'Alice', 25, 'New York', 250.50])
9     csv_writer.writerow([2, 'Bob', 30, 'Los Angeles', 180.00])
10    csv_writer.writerow([3, 'Charlie', 35, 'Chicago', 320.75])
11
12 print("Rows written successfully!")

```

2.3.3 CSV DictWriter (Write from Dictionaries)

```

1 import csv
2
3 data = [
4     {'customer_id': 1, 'name': 'Alice', 'age': 25, 'city': 'New York', 'amount': 250.50},
5     {'customer_id': 2, 'name': 'Bob', 'age': 30, 'city': 'Los Angeles', 'amount': 180.00},
6     {'customer_id': 3, 'name': 'Charlie', 'age': 35, 'city': 'Chicago', 'amount': 320.75}
7 ]
8
9 fieldnames = ['customer_id', 'name', 'age', 'city', 'amount']
10
11 with open('output_dictwriter.csv', 'w', newline='') as file:
12     writer = csv.DictWriter(file, fieldnames=fieldnames)
13
14     writer.writeheader()
15     writer.writerows(data)
16
17 print("DictWriter CSV created successfully!")

```

2.3.4 Writing with Different Delimiter

```

1 import csv
2
3 data = [
4     ['Name', 'City', 'Amount'],
5     ['Alice', 'New York', 250.50],
6     ['Bob', 'Los Angeles', 180.00]
7 ]
8
9 # Semicolon delimiter
10 with open('output_semicolon.csv', 'w', newline='') as file:
11     csv_writer = csv.writer(file, delimiter=';')
12     csv_writer.writerows(data)
13
14 # Tab delimiter
15 with open('output_tab.tsv', 'w', newline='') as file:
16     csv_writer = csv.writer(file, delimiter='\t')
17     csv_writer.writerows(data)
18
19 print("Files with different delimiters created!")

```

2.3.5 Appending to Existing CSV File

```

1 import csv
2
3 new_rows = [
4     [6, 'Frank', 38, 'San Antonio', 200.00],
5     [7, 'Grace', 27, 'San Diego', 450.00]
6 ]
7
8 with open('customers.csv', 'a', newline='') as file:
9     csv_writer = csv.writer(file)
10    csv_writer.writerows(new_rows)
11
12 print("Data appended successfully!")

```

3. Method 2: Using Pandas for CSV Operations

Pandas is more powerful and recommended for data analysis. It requires installation (`pip install pandas`).

3.1 Reading CSV Files with Pandas

```
1 import pandas as pd
2
3 # Basic read
4 df = pd.read_csv('customers.csv')
5 print(df)
```

Output:

	customer_id	name	age	city	amount
0	1	Alice	25	New York	250.50
1	2	Bob	30	Los Angeles	180.00
2	3	Charlie	35	Chicago	320.75
3	4	Diana	28	Houston	95.25
4	5	Eve	42	Phoenix	510.00

3.2 Common Parameters of `pd.read_csv()`

```
1 df = pd.read_csv(
2     'customers.csv',           # File path
3     sep=',',                  # Delimiter (default is comma)
4     header=0,                 # Row number for column names
5     index_col='customer_id',  # Use column as index
6     usecols=['name', 'age', 'city'], # Select specific columns
7     nrows=3,                  # Read only first 3 rows
8     skiprows=[1, 3],          # Skip rows
9     na_values=['', 'NA', 'NULL'], # Missing values
10    dtype={'age': int, 'amount': float}, # Data types
11    encoding='utf-8'           # File encoding
12 )
13
14 print(df)
```

3.3 Reading CSV Without Headers

```
1 df = pd.read_csv(
2     'customers_no_header.csv',
3     names=['id', 'name', 'age', 'city', 'amount']
4 )
5
6 print(df)
```

3.4 Reading Large CSV Files in Chunks

```
1 chunk_size = 1000
2
3 for chunk in pd.read_csv('large_file.csv', chunksize=chunk_size):
4     print(f"Processing chunk with {len(chunk)} rows")
```

3.5 Writing CSV Files with Pandas

```
1 import pandas as pd
2
3 data = {
4     'customer_id': [1, 2, 3],
5     'name': ['Alice', 'Bob', 'Charlie'],
6     'age': [25, 30, 35],
7     'city': ['New York', 'Los Angeles', 'Chicago'],
8     'amount': [250.50, 180.00, 320.75]
9 }
10
11 df = pd.DataFrame(data)
12
13 df.to_csv('output_pandas.csv', index=False)
14 print("CSV file saved!")
```

3.6 Common Parameters of `df.to_csv()`

```
1 df.to_csv(
2     'output_customers.csv',
3     sep=',',
4     index=False,
5     header=True,
6     columns=['name', 'age', 'city'],
7     encoding='utf-8',
```



```
8     na_rep='NULL',
9     float_format='%.2f'
10 )
```

3.7 Appending with Pandas

```
1 new_data = pd.DataFrame({
2     'customer_id': [6, 7],
3     'name': ['Frank', 'Grace'],
4     'age': [38, 27],
5     'city': ['San Antonio', 'San Diego'],
6     'amount': [200.00, 450.00]
7 })
8
9 new_data.to_csv(
10     'customers.csv',
11     mode='a',
12     header=False,
13     index=False
14 )
```

4. Complete Code Examples (CSV Module + Pandas Together)

Example 1: CSV Reader – Read and Process

```
1 import csv
2
3 print("=" * 50)
4 print("USING CSV MODULE - READER")
5 print("=" * 50)
6
7 with open('customers.csv', 'r') as file:
8     csv_reader = csv.reader(file)
9     headers = next(csv_reader)
10    print(f"Headers: {headers}")
11
12    customers = []
13    for row in csv_reader:
14        customers.append({
15            'id': int(row[0]),
16            'name': row[1],
17            'age': int(row[2]),
18            'city': row[3],
19            'amount': float(row[4])
20        })
21
22    print(f"\nTotal customers: {len(customers)}")
23    print("First customer:", customers[0])
```

Example 2: CSV DictReader – Read as Dictionary

```
1 import csv
2
3 print("=" * 50)
4 print("USING CSV MODULE - DICTREADER")
5 print("=" * 50)
6
7 with open('customers.csv', 'r') as file:
8     csv_reader = csv.DictReader(file)
9
10    customers = list(csv_reader)
11
12    for customer in customers[:3]:
13        print(f"Name: {customer['name']}, City: {customer['city']}, Amount: ${customer['amount']}")
```

Example 3: CSV Writer – Write Data

```
1 import csv
2
3 print("=" * 50)
4 print("USING CSV MODULE - WRITER")
5 print("=" * 50)
6
7 data = [
8     ['customer_id', 'name', 'age', 'city', 'amount'],
9     [1, 'Alice', 25, 'New York', 250.50],
10    [2, 'Bob', 30, 'Los Angeles', 180.00],
11    [3, 'Charlie', 35, 'Chicago', 320.75],
12    [4, 'Diana', 28, 'Houston', 95.25],
13    [5, 'Eve', 42, 'Phoenix', 510.00]
14 ]
```



```
15
16 with open('new_customers.csv', 'w', newline='') as file:
17     csv_writer = csv.writer(file)
18     csv_writer.writerows(data)
19
20 print("File created successfully!")
```

Example 4: CSV DictWriter – Write from Dictionaries

```
1 import csv
2
3 print("=" * 50)
4 print("USING CSV MODULE - DICTWRITER")
5 print("=" * 50)
6
7 customers = [
8     {'id': 1, 'name': 'Alice', 'amount': 250.50},
9     {'id': 2, 'name': 'Bob', 'amount': 180.00},
10    {'id': 3, 'name': 'Charlie', 'amount': 320.75}
11 ]
12
13 fieldnames = ['id', 'name', 'amount']
14
15 with open('dictwriter_output.csv', 'w', newline='') as file:
16     writer = csv.DictWriter(file, fieldnames=fieldnames)
17     writer.writeheader()
18     writer.writerows(customers)
19
20 print("DictWriter output saved!")
```

Example 5: Pandas CSV Operations

```
1 import pandas as pd
2
3 print("=" * 50)
4 print("USING PANDAS FOR CSV")
5 print("=" * 50)
6
7 df = pd.read_csv('customers.csv')
8 print(df)
9
10 df['discount'] = df['amount'] * 0.10
11 df['final_amount'] = df['amount'] - df['discount']
12
13 high_value = df[df['amount'] > 200]
14 print(high_value[['name', 'amount', 'final_amount']])
15
16 high_value.to_csv('high_value_customers.csv', index=False)
17 print("Saved successfully!")
```

Example 6: Complete Mixed Example (Read → Process → Write)

```
1 import csv
2 import pandas as pd
3
4 print("=" * 60)
5 print("COMPLETE EXAMPLE: READ PROCESS WRITE")
6 print("=" * 60)
7
8 print("\n[1] Reading CSV using csv.reader...")
9 with open('customers.csv', 'r') as file:
10     reader = csv.reader(file)
11     headers = next(reader)
12     rows = list(reader)
13
14 print(f"Read {len(rows)} rows")
15
16 print("\n[2] Processing data...")
17 processed_data = []
18 for row in rows:
19     customer = {
20         'customer_id': int(row[0]),
21         'name': row[1],
22         'age': int(row[2]),
23         'city': row[3],
24         'amount': float(row[4])
25     }
26     customer['discount'] = customer['amount'] * 0.10
27     customer['final_amount'] = customer['amount'] - customer['discount']
28     processed_data.append(customer)
29
30 print(f"Processed {len(processed_data)} records")
```

```
31
32 print("\n[3] Writing processed data...")
33 with open('processed_customers.csv', 'w', newline='') as file:
34     fieldnames = ['customer_id', 'name', 'age', 'city', 'amount', 'discount', 'final_amount']
35     writer = csv.DictWriter(file, fieldnames=fieldnames)
36     writer.writeheader()
37     writer.writerows(processed_data)
38
39 print("Saved successfully!")
40
41 print("\n[4] Verifying with pandas...")
42 df = pd.read_csv('processed_customers.csv')
43 print(df.head())
44 print(f"Shape: {df.shape}")
```

5. Comparison: CSV Module vs Pandas

Feature	csv module (built-in)	pandas
Installation	No installation needed	Requires <code>pip install pandas</code>
Speed	Slower for large files	Faster (optimized in C)
Memory usage	Low (line-by-line processing)	High (loads full dataset)
Data types	All values as strings	Automatic type inference
Missing values	Manual handling required	Built-in NaN support
Filtering	Manual loops	Simple indexing
Aggregations	Manual coding	Built-in functions (groupby, mean, sum)
Writing files	csv.writer	df.to_csv()
Best for	Simple I/O, large streaming files	Data analysis, ML, transformations

Working with APIs for Data Collection

1. What is an API?

API (Application Programming Interface) is a set of rules and protocols that allows one software application to communicate with another. It acts as a bridge between different software systems, enabling them to exchange data and functionality.

1.1 Simple Analogy

Real-World Analogy	API Analogy
Restaurant	API
You (customer)	Your application (client)
Waiter	API
Kitchen	Server/database
Menu	API documentation (endpoints)
Food	Data/response

How it works:

- You look at the menu (API documentation)
- You send a request through the waiter (API)
- Waiter communicates with the kitchen (server)
- Kitchen processes the request
- Waiter returns the response (food/data)

1.2 Why APIs are Important in Data Science

Use Case	Example
Collect real-time data	Live stock prices, weather data, sports scores
Access third-party data	Twitter feeds, Google Maps, YouTube analytics
Send predictions	Deploy ML model as API for other apps
Automate data collection	Scheduled API calls for regular updates
Integrate multiple sources	Combine CRM, payment, and support systems

1.3 Key API Terminology

Term	Definition	Example
Endpoint	Specific URL to access API	https://api.weather.com/v1/current
Request	Message sent to server	GET /users/123
Response	Data returned by server	{"name": "Alice", "age": 25}
Method/Verb	Action performed	GET, POST, PUT, DELETE
Header	Metadata for request	API key, content type
Body	Data sent with request	JSON payload
Status Code	Server response status	200, 404, 500
Authentication	User verification	API Key, OAuth Token

2. HTTP Methods (Request Verbs)

Method	Purpose	Analogy	Example
GET	Retrieve data	Asking for information	Get customer details
POST	Create new data	Submitting a form	Create new customer
PUT	Update entire record	Replacing a file	Update full customer info
PATCH	Update partial record	Editing a field	Update phone number
DELETE	Remove data	Deleting a file	Delete customer

2.1 HTTP Status Codes

Range	Category	Common Codes
2xx	Success	200 OK, 201 Created
3xx	Redirection	301 Moved Permanently
4xx	Client Error	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Server Error	500 Internal Server Error, 503 Service Unavailable

3. REST API (Representational State Transfer)

3.1 What is REST API?

REST (Representational State Transfer) is an architectural style for designing networked applications. REST APIs follow constraints that make them simple, scalable, and lightweight.

3.2 Principles of REST API

Principle	Explanation
Stateless	Each request contains all required information; server does not store session
Client-Server	Client and server operate independently
Cacheable	Responses can be cached for performance
Uniform Interface	Standardized communication using HTTP methods
Layered System	Client may interact through intermediaries

3.3 REST API Characteristics

Characteristic	Description
Uses HTTP	Built on standard HTTP protocol
Data Format	Usually JSON (lightweight, readable)
Stateless	No memory of previous requests
Simple	Easy to implement and understand
Scalable	Handles large number of requests

3.4 REST API Example

Endpoint: `https://jsonplaceholder.typicode.com/users`

Request:

```
1 GET https://jsonplaceholder.typicode.com/users
```

Response (JSON):

```
1 [
2   {
3     "id": 1,
4     "name": "Leanne Graham",
5     "username": "Bret",
6     "email": "Sincere@april.biz",
7     "address": {
8       "street": "Kulas Light",
9       "city": "Gwenborough"
10    }
11  }
12 ]
```

3.5 REST API Request Components

```
1 GET https://api.example.com/users/123?include=orders&limit=10 HTTP/1.1
2 Host: api.example.com
3 Authorization: Bearer xyz123token
4 Content-Type: application/json
```

Component	Value	Purpose
Method	GET	Action to perform
Base URL	https://api.example.com	Root API address
Endpoint	/users/123	Specific resource
Query Parameters	?include=orders&limit=10	Filter response
Headers	Authorization, Content-Type	Metadata
Body	None (GET request)	Data sent (POST/PUT only)

4. SOAP API (Simple Object Access Protocol)

4.1 What is SOAP?

SOAP (Simple Object Access Protocol) is a protocol for exchanging structured information in web services. It is more rigid and formal than REST.

4.2 SOAP Characteristics

Characteristic	Description
Protocol	Strict protocol with formal rules
Data Format	XML only (heavier than JSON)
Transport	HTTP, SMTP, TCP, etc.
Stateful/Stateless	Can be stateful (remembers session)
Error Handling	Standardized fault messages
Security	Built-in WS-Security
Complexity	More complex to implement

4.3 SOAP Message Structure

```
1 <?xml version="1.0"?>
2 <soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
3
4   <soap:Header>
5     <auth:Authentication xmlns:auth="http://example.com/auth">
6       <auth:apiKey>xyz123</auth:apiKey>
7     </auth:Authentication>
8   </soap:Header>
9
10  <soap:Body>
11    <GetUserInfo xmlns="http://example.com/user">
12      <UserID>12345</UserID>
13    </GetUserInfo>
14  </soap:Body>
15
16 </soap:Envelope>
```

Element	Purpose
Envelope	Root element identifying SOAP message
Header	Optional metadata (authentication, routing)
Body	Contains actual request or response
Fault	Error information inside Body

4.4 SOAP Example Response

```
1 <?xml version="1.0"?>
2 <soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
3
4   <soap:Body>
5     <GetUserInfoResponse xmlns="http://example.com/user">
6       <UserInfo>
7         <ID>12345</ID>
8         <Name>John Doe</Name>
9         <Email>john@example.com</Email>
10      </UserInfo>
11    </GetUserInfoResponse>
12  </soap:Body>
13
14 </soap:Envelope>
```

5. REST API vs SOAP API (Comparison)

Feature	REST API	SOAP API
Full Form	Representational State Transfer	Simple Object Access Protocol
Type	Architectural style	Protocol
Data Format	JSON, XML, HTML, Plain text	XML only
Message Size	Small (lightweight)	Large (verbose XML)
Speed	Faster	Slower
State	Stateless (always)	Can be stateful or stateless
Caching	Supported	Not supported
Security	HTTPS, API Keys, OAuth	WS-Security (built-in)
Error Handling	HTTP status codes	SOAP Faults
Complexity	Simple, easy to learn	Complex, steep learning curve
Human Readable	Yes (JSON readable)	No (XML verbose)
Browser Support	Excellent	Limited
Best For	Web and mobile apps, public APIs	Enterprise systems, banking, payments
Examples	Google Maps API, Twitter API	PayPal SOAP API, banking systems

5.1 When to Use REST API?

- Web and mobile application development
- Public APIs for external developers
- Fast performance with lightweight communication
- JSON-based data exchange
- Stateless operations

5.2 When to Use SOAP API?

- Enterprise-level systems (banking, healthcare)
- High security and transaction requirements
- Formal contracts using WSDL
- Stateful operations required
- Integration with legacy systems

6. Working with REST APIs in Python

6.1 Installing Required Library

```
1 pip install requests
```

6.2 Importing Requests Module

```
1 import requests
```

6.3 GET Request (Retrieve Data)

Basic GET Request

```
1 import requests
2
3 response = requests.get('https://jsonplaceholder.typicode.com/users')
4
5 print(f"Status Code: {response.status_code}")
6 print(f"Response Type: {type(response.json())}")
7
8 users = response.json()
9 print(f"Number of users: {len(users)}")
10 print(f"First user: {users[0]['name']}")
```

GET Request with Query Parameters

```
1 import requests
2
3 params = {
4     'postId': 1,
5     '_limit': 3
6 }
7
8 response = requests.get(
9     'https://jsonplaceholder.typicode.com/comments',
10    params=params
11 )
12
13 print(f"URL that was called: {response.url}")
14 comments = response.json()
15
16 for comment in comments:
17     print(f"Comment {comment['id']}: {comment['name'][:50]}...")
```

GET Request with Headers (Authentication)

```
1 import requests
2
3 headers = {
4     'Authorization': 'Bearer YOUR_API_TOKEN',
5     'Content-Type': 'application/json',
6     'User-Agent': 'MyDataScienceApp/1.0'
7 }
8
9 response = requests.get(
10    'https://api.example.com/data',
11    headers=headers
12 )
13
14 if response.status_code == 200:
15     data = response.json()
16     print("Data retrieved successfully")
17 else:
18     print(f"Error: {response.status_code}")
```

6.4 POST Request (Create Data)

```
1 import requests
2
3 new_user = {
4     'name': 'Alice Johnson',
5     'username': 'alicej',
6     'email': 'alice@example.com',
7     'phone': '555-1234'
8 }
9
10 response = requests.post(
11    'https://jsonplaceholder.typicode.com/users',
12    json=new_user
13 )
14
15 print(f"Status Code: {response.status_code}")
16 print(f"Response: {response.json()}")
```

6.5 PUT Request (Update Entire Record)

```
1 import requests
2
3 updated_user = {
4     'id': 1,
5     'name': 'Leanne Graham Updated',
6     'username': 'Bret_UPDATED',
7     'email': 'updated@example.com'
8 }
9
10 response = requests.put(
11    'https://jsonplaceholder.typicode.com/users/1',
12    json=updated_user
13 )
14
15 print(f"Status Code: {response.status_code}")
16 print(f"Updated User: {response.json()}")
```


6.6 DELETE Request (Remove Data)

```
1 import requests
2
3 response = requests.delete(
4     'https://jsonplaceholder.typicode.com/users/1'
5 )
6
7 print(f"Status Code: {response.status_code}")
8
9 if response.status_code == 200:
10     print("User deleted successfully")
11 else:
12     print("Deletion failed")
```

6.7 Error Handling with APIs

```
1 import requests
2
3 url = "https://jsonplaceholder.typicode.com/users/999"
4
5 try:
6     response = requests.get(url, timeout=5)
7     response.raise_for_status()
8
9     data = response.json()
10    print("Data:", data)
11
12 except requests.exceptions.HTTPError as e:
13     print(f"HTTP Error: {e}")
14     print(f"Status Code: {response.status_code}")
15
16 except requests.exceptions.ConnectionError:
17     print("Failed to connect to server.")
18
19 except requests.exceptions.Timeout:
20     print("Request timed out.")
21
22 except requests.exceptions.RequestException as e:
23     print(f"An error occurred: {e}")
```

7. Real-World API Examples for Data Science

Example 1: Weather API (OpenWeatherMap)

```
1 import requests
2
3 API_KEY = "your_api_key_here"
4 CITY = "London"
5
6 url = f"http://api.openweathermap.org/data/2.5/weather?q={CITY}&appid={API_KEY}&units=metric"
7
8 response = requests.get(url)
9
10 if response.status_code == 200:
11     weather_data = response.json()
12
13     print(f"City: {weather_data['name']}")
14     print(f"Temperature: {weather_data['main']['temp']}C")
15     print(f"Humidity: {weather_data['main']['humidity']}%")
16     print(f"Conditions: {weather_data['weather'][0]['description']}")
17 else:
18     print(f"Error: {response.status_code}")
```

Example 2: News API (Fetch Latest Headlines)

```
1 import requests
2
3 API_KEY = "your_api_key_here"
4 url = f"https://newsapi.org/v2/top-headlines?country=us&apiKey={API_KEY}"
5
6 response = requests.get(url)
7
8 if response.status_code == 200:
9     news = response.json()
10
11     print(f"Total articles: {news['totalResults']}")
12     print("\nTop 5 Headlines:")
13
14     for i, article in enumerate(news['articles'][:5], 1):
```



```
15     print(f"{i}. {article['title']}")
16     print(f"    Source: {article['source']['name']}\n")
```

Example 3: GitHub API (Fetch User Repositories)

```
1 import requests
2
3 username = "octocat"
4 url = f"https://api.github.com/users/{username}/repos"
5
6 response = requests.get(url)
7
8 if response.status_code == 200:
9     repos = response.json()
10
11     print(f"Repositories for {username}:")
12     for repo in repos[:5]:
13         print(f"    - {repo['name']}: {repo['stargazers_count']} stars")
14 else:
15     print(f"Error: {response.status_code}")
```

8. Working with SOAP APIs in Python

SOAP APIs are less common but still used in enterprise environments.

8.1 Installing Required Library

```
1 pip install zeep
```

8.2 Basic SOAP Request

```
1 from zeep import Client
2
3 wsdl_url = "http://www.dneonline.com/calculator.asmx?wsdl"
4
5 client = Client(wsdl_url)
6
7 result = client.service.Add(intA=10, intB=20)
8
9 print(f"10 + 20 = {result}")
```

8.3 SOAP with Headers and Authentication

```
1 from zeep import Client
2 from zeep.wsse.username import UsernameToken
3
4 client = Client(
5     'https://example.com/service.asmx?wsdl',
6     wsse=UsernameToken('username', 'password')
7 )
8
9 response = client.service.GetUserDetails(UserID=12345)
10 print(response)
```

9. Complete API Data Collection Pipeline

```
1 import requests
2 import pandas as pd
3 import time
4
5 print("=" * 60)
6 print("COMPLETE API DATA COLLECTION PIPELINE")
7 print("=" * 60)
8
9 API_URL = "https://jsonplaceholder.typicode.com/posts"
10 params = {
11     '_limit': 20
12 }
13
14 print("\n[1] Fetching data from API...")
15 try:
16     response = requests.get(API_URL, params=params, timeout=10)
17     response.raise_for_status()
18     print(f"    Status: {response.status_code} - Success")
19 except Exception as e:
20     print(f"    Error: {e}")
21 exit()
```

```
22
23 print("\n[2] Parsing JSON response...")
24 data = response.json()
25 print(f"    Retrieved {len(data)} records")
26
27 print("\n[3] Converting to pandas DataFrame...")
28 df = pd.DataFrame(data)
29 print(f"    Shape: {df.shape}")
30 print(f"    Columns: {list(df.columns)}")
31
32 print("\n[4] Processing data...")
33
34 df['title_length'] = df['title'].str.len()
35 df['body_word_count'] = df['body'].str.split().str.len()
36
37 print("    Added features:")
38 print("        - title_length")
39 print("        - body_word_count")
40
41 print("\n[5] Saving to CSV...")
42 csv_filename = "api_posts_data.csv"
43 df.to_csv(csv_filename, index=False)
44 print(f"    Saved to '{csv_filename}'")
45
46 print("\n[6] Data Preview:")
47 print(df[['id', 'title', 'title_length', 'body_word_count']].head())
48
49 print("\n" + "=" * 60)
50 print("API DATA COLLECTION COMPLETED SUCCESSFULLY!")
51 print("=" * 60)
```

10. Best Practices for Working with APIs

Best Practice	Explanation
Store API keys securely	Use environment variables, never hardcode keys
Handle rate limits	Add delays between requests to avoid blocking
Implement retry logic	Retry failed requests using exponential backoff
Cache responses	Store API data locally to reduce repeated calls
Validate response data	Check status codes and structure before processing
Use timeouts	Prevent hanging requests using timeout limits
Log API calls	Maintain logs for debugging and monitoring
Respect terms of service	Follow API provider usage policies

Secure API Key Storage Example

```
1 import os
2 from dotenv import load_dotenv
3
4 load_dotenv()
5
6 API_KEY = os.getenv('WEATHER_API_KEY')
7
8 # Never hardcode API keys like:
9 # API_KEY = "abc123"
```

11. Quick Revision Summary

Concept	Key Points
API	Interface allowing software systems to communicate
REST API	Lightweight, uses JSON, stateless, based on HTTP methods
SOAP API	Heavy, XML-based, protocol-driven, can be stateful
GET	Retrieve data (read operation)
POST	Create new data
PUT	Update entire record
DELETE	Remove data
Status Code 200	OK – Request successful
Status Code 201	Created – Resource successfully created
Status Code 400	Bad Request – Client-side error
Status Code 401	Unauthorized – Authentication required
Status Code 404	Not Found – Endpoint does not exist
Status Code 500	Internal Server Error – Server-side failure
Endpoint	Specific URL where API is accessed
Header	Metadata sent with request (e.g., authentication, content type)
Query Parameter	Key-value pair in URL used for filtering (?key=value)

Data Cleaning: Handling Missing Values and Duplicates

1. What is Data Cleaning?

Data Cleaning (or Data Cleansing) is the process of detecting and correcting (or removing) corrupt, inaccurate, incomplete, or duplicate records from a dataset.

1.1 Importance of Data Cleaning

Reason	Explanation
Improves accuracy	Clean data leads to reliable analysis and predictions
Saves time	Prevents errors that waste time later in the pipeline
Increases productivity	Analysts trust data and work faster
Reduces storage	Removing duplicates and irrelevant data saves space
Better decisions	Garbage in = Garbage out (GIGO principle)

1.2 Common Data Quality Issues

Issue	Description	Example
Missing Values	Empty or null values in dataset	Age column has NaN
Duplicate Records	Same record appears multiple times	Same customer listed twice
Outliers	Extreme values outside normal range	Age = 999 years
Inconsistent Formatting	Same data stored differently	“NY”, “New York”, “new york”
Incorrect Data Types	Wrong type for the column	Price stored as text “\$49.99”
Invalid Values	Impossible values	Gender = “Unknown123”
Contradictions	Logically inconsistent data	Birth date = 2000, Age = 50

2. Handling Missing Values

2.1 What are Missing Values?

Missing values are empty, null, or NA (Not Available) entries in a dataset. Common representations:

Representation	Description
NaN	Not a Number (pandas default)
None	Python’s null value
null	JSON/JavaScript null
NA	Not Available
? or -	Placeholder characters
Empty string ""	Blank value
0 or -1	Placeholder numbers (dangerous)

2.2 Types of Missing Data Patterns

Pattern	Abbreviation	Meaning	Example
Missing Completely at Random	MCAR	No relationship between missingness and any data	Random sensor glitch
Missing at Random	MAR	Missingness depends on other observed variables	Men skip pregnancy question
Missing Not at Random	MNAR	Missingness depends on the missing value itself	High-income people hide income

2.3 Step 1: Detect Missing Values

Using CSV Module:

```
1 import csv
2
3 def detect_missing_csv(filename):
4     """Detect missing values in CSV file using csv module"""
5
6     with open(filename, 'r') as file:
7         reader = csv.DictReader(file)
8         rows = list(reader)
9
10        # Get column names
11        columns = reader.fieldnames
12
13        # Initialize missing count dictionary
14        missing_counts = {col: 0 for col in columns}
15
```

```
16     # Count missing values
17     for row in rows:
18         for col in columns:
19             if not row[col] or row[col].strip() == '' or row[col] == 'NA' or row[col] == 'null':
20                 missing_counts[col] += 1
21
22     # Print results
23     total_rows = len(rows)
24     print(f"Total rows: {total_rows}")
25     print("\nMissing Values Report:")
26     print("-" * 40)
27     for col, count in missing_counts.items():
28         percentage = (count / total_rows) * 100
29         print(f"{col:15} | Missing: {count:5} | {percentage:5.1f}%")
30
31     return missing_counts, total_rows
32
33 # Usage
34 detect_missing_csv('customers.csv')
```

Listing 2.1: Detect Missing Values using CSV Module

Sample Output:

```
1 Total rows: 100
2
3 Missing Values Report:
4 -----
5 customer_id    | Missing:    0 |    0.0%
6 name          | Missing:    2 |    2.0%
7 age           | Missing:    5 |    5.0%
8 city          | Missing:    3 |    3.0%
9 amount        | Missing:    1 |    1.0%
```

Using Pandas

```
1 import pandas as pd
2
3 # Read CSV
4 df = pd.read_csv('customers.csv')
5
6 # Check missing values
7 print("Missing Values Count:")
8 print(df.isnull().sum())
9
10 print("\nMissing Values Percentage:")
11 print((df.isnull().sum() / len(df)) * 100)
12
13 # Visual check of missing values
14 print("\nDataFrame Info:")
15 print(df.info())
16
17 # Check for specific missing representations
18 df = pd.read_csv('customers.csv', na_values=['', 'NA', 'NULL', '?', 'missing'])
```

Listing 2.2: Detect Missing Values using Pandas

2.4 Step 2: Understand Missing Data Pattern

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Check which rows have missing values
6 rows_with_missing = df[df.isnull().any(axis=1)]
7 print(f"Rows with at least one missing value: {len(rows_with_missing)}")
8
9 # Check rows with all missing values
10 rows_all_missing = df[df.isnull().all(axis=1)]
11 print(f"Rows with all missing values: {len(rows_all_missing)}")
12
13 # Check missing pattern across columns
14 missing_matrix = df.isnull().sum()
15 print(missing_matrix)
16
17 # Visualize missing values (if matplotlib available)
18 try:
19     import missingno as msno
20     msno.matrix(df)
21     plt.show()
22 except:
23     print("missingno library not installed. Skipping visualization.")
```

Listing 2.3: Understanding Missing Data Patterns

2.5 Step 3: Handle Missing Values

Method A: Deletion Methods

A1. Delete Rows with Missing Values (Listwise Deletion) Using Pandas:

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Delete rows with any missing value
6 df_cleaned = df.dropna()
7 print(f"Original: {len(df)} rows")
8 print(f"After dropna(): {len(df_cleaned)} rows")
9
10 # Delete rows with missing values in specific columns
11 df_cleaned = df.dropna(subset=['age', 'amount'])
12 print(f"After dropping rows missing age or amount: {len(df_cleaned)} rows")
13
14 # Delete rows where all values are missing
15 df_cleaned = df.dropna(how='all')
16
17 # Delete rows where at least 3 values are missing
18 df_cleaned = df.dropna(thresh=len(df.columns) - 2)
```

Listing 2.4: Drop Missing Rows using Pandas

Using CSV Module:

```
1 import csv
2
3 def drop_missing_rows_csv(input_file, output_file, columns_to_check=None):
4     """Delete rows with missing values from CSV"""
5
6     with open(input_file, 'r') as infile:
7         reader = csv.DictReader(infile)
8         fieldnames = reader.fieldnames
9         rows = list(reader)
10
11     original_count = len(rows)
12
13     # Filter rows
14     if columns_to_check:
15         filtered_rows = [row for row in rows
16                         if all(row[col] and row[col].strip() for col in columns_to_check)]
17     else:
18         filtered_rows = [row for row in rows
19                         if all(row[col] and row[col].strip() for col in fieldnames)]
20
21     removed_count = original_count - len(filtered_rows)
22
23     # Write cleaned data
24     with open(output_file, 'w', newline='') as outfile:
25         writer = csv.DictWriter(outfile, fieldnames=fieldnames)
26         writer.writeheader()
27         writer.writerows(filtered_rows)
28
29     print(f"Original rows: {original_count}")
30     print(f"Removed rows: {removed_count}")
31     print(f"Remaining rows: {len(filtered_rows)}")
32
33     return filtered_rows
34
35 # Usage
36 drop_missing_rows_csv('customers.csv', 'customers_no_missing.csv', ['age', 'amount'])
```

Listing 2.5: Drop Missing Rows using CSV Module

A2. Delete Columns with Many Missing Values Using Pandas:

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Drop columns with > 50% missing values
6 threshold = 0.5
7 missing_percentage = df.isnull().sum() / len(df)
8 cols_to_drop = missing_percentage[missing_percentage > threshold].index
9 df_cleaned = df.drop(columns=cols_to_drop)
10
11 print(f"Dropped columns: {list(cols_to_drop)}")
12
13 # Drop column entirely
14 df_cleaned = df.drop(columns=['unnecessary_column'])
```

Listing 2.6: Drop Columns with Missing Values using Pandas

Using CSV Module:

```
1 import csv
2
3 def drop_columns_csv(input_file, output_file, columns_to_drop):
4     """Delete specific columns from CSV"""
5
6     with open(input_file, 'r') as infile:
7         reader = csv.DictReader(infile)
8         fieldnames = [col for col in reader.fieldnames if col not in columns_to_drop]
9
10    with open(output_file, 'w', newline='') as outfile:
11        writer = csv.DictWriter(outfile, fieldnames=fieldnames)
12        writer.writeheader()
13
14        for row in reader:
15            for col in columns_to_drop:
16                del row[col]
17            writer.writerow(row)
18
19    print(f"Removed columns: {columns_to_drop}")
20    print(f"Remaining columns: {fieldnames}")
21
22 # Usage
23 drop_columns_csv('customers.csv', 'customers_no_age.csv', ['age'])
```

Listing 2.7: Drop Columns using CSV Module

Method B: Imputation Methods (Filling Missing Values)

B1. Mean Imputation (For Numeric Columns) Using Pandas:

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Calculate mean
6 mean_age = df['age'].mean()
7 print(f"Mean age: {mean_age:.2f}")
8
9 # Fill missing values with mean
10 df['age_filled'] = df['age'].fillna(mean_age)
11
12 # Alternative: fill in place
13 df['age'].fillna(df['age'].mean(), inplace=True)
```

Listing 2.8: Mean Imputation using Pandas

Using CSV Module:

```
1 import csv
2
3 def mean_imputation_csv(input_file, output_file, numeric_column):
4     """Fill missing numeric values with column mean"""
5
6     with open(input_file, 'r') as infile:
7         reader = csv.DictReader(infile)
8         rows = list(reader)
9
10    # Calculate mean
11    values = []
12    for row in rows:
13        if row[numeric_column] and row[numeric_column].strip():
14            try:
15                values.append(float(row[numeric_column]))
16            except ValueError:
17                pass
18
19    mean_value = sum(values) / len(values) if values else 0
20
21    # Fill missing values
22    for row in rows:
23        if not row[numeric_column] or row[numeric_column].strip() == '':
24            row[numeric_column] = str(mean_value)
25
26    # Write to output
27    with open(output_file, 'w', newline='') as outfile:
28        writer = csv.DictWriter(outfile, fieldnames=reader.fieldnames)
29        writer.writeheader()
30        writer.writerows(rows)
31
32    print(f"Missing values in '{numeric_column}' filled with mean: {mean_value:.2f}")
33
34 # Usage
35 mean_imputation_csv('customers.csv', 'customers_mean_imputed.csv', 'age')
```

Listing 2.9: Mean Imputation using CSV Module

B2. Median Imputation (For Skewed Numeric Columns)

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Calculate median
6 median_age = df['age'].median()
7 median_amount = df['amount'].median()
8
9 # Fill with median (better for skewed data with outliers)
10 df['age'].fillna(df['age'].median(), inplace=True)
11 df['amount'].fillna(df['amount'].median(), inplace=True)
12
13 print(f"Filled age with median: {median_age}")
14 print(f"Filled amount with median: {median_amount}")
```

Listing 2.10: Median Imputation using Pandas

B3. Mode Imputation (For Categorical Columns) Using Pandas:

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Calculate mode (most frequent value)
6 mode_city = df['city'].mode()[0]
7 print(f"Most frequent city: {mode_city}")
8
9 # Fill with mode
10 df['city'].fillna(mode_city, inplace=True)
```

Listing 2.11: Mode Imputation using Pandas

Using CSV Module:

```
1 import csv
2 from collections import Counter
3
4 def mode_imputation_csv(input_file, output_file, categorical_column):
5     """Fill missing categorical values with column mode"""
6
7     with open(input_file, 'r') as infile:
8         reader = csv.DictReader(infile)
9         rows = list(reader)
10
11     # Find mode (most frequent value)
12     valid_values = []
13     for row in rows:
14         if row[categorical_column] and row[categorical_column].strip():
15             valid_values.append(row[categorical_column])
16
17     if valid_values:
18         mode_value = Counter(valid_values).most_common(1)[0][0]
19     else:
20         mode_value = "Unknown"
21
22     # Fill missing values
23     for row in rows:
24         if not row[categorical_column] or row[categorical_column].strip() == '':
25             row[categorical_column] = mode_value
26
27     # Write to output
28     with open(output_file, 'w', newline='') as outfile:
29         writer = csv.DictWriter(outfile, fieldnames=reader.fieldnames)
30         writer.writeheader()
31         writer.writerows(rows)
32
33     print(f"Missing values in '{categorical_column}' filled with mode: {mode_value}")
34
35 # Usage
36 mode_imputation_csv('customers.csv', 'customers_mode_imputed.csv', 'city')
```

Listing 2.12: Mode Imputation using CSV Module

B4. Constant Imputation

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Fill with specific constant value
6 df['city'].fillna('Unknown', inplace=True)
7 df['age'].fillna(-1, inplace=True) # Placeholder for missing
8 df['amount'].fillna(0, inplace=True)
9
10 # Create indicator column for missing values
11 df['age_missing'] = df['age'].isnull().astype(int)
```


Listing 2.13: Constant Imputation using Pandas

B5. Forward Fill and Backward Fill (For Time Series)

```
1 import pandas as pd
2
3 # Example time series data
4 df_time = pd.DataFrame({
5     'date': ['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04'],
6     'value': [100, None, None, 150]
7 })
8
9 # Forward fill
10 df_time['value_ffill'] = df_time['value'].fillna(method='ffill')
11
12 # Backward fill
13 df_time['value_bfill'] = df_time['value'].fillna(method='bfill')
14
15 # Interpolation
16 df_time['value_interp'] = df_time['value'].interpolate()
17
18 print(df_time)
```

Listing 2.14: Forward Fill

Output:

	date	value	value_ffill	value_bfill	value_interp
0	2024-01-01	100.0	100.0	100.0	100.0
1	2024-01-02	NaN	100.0	150.0	116.7
2	2024-01-03	NaN	100.0	150.0	133.3
3	2024-01-04	150.0	150.0	150.0	150.0

B6. KNN Imputation (Advanced)

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.impute import KNNImputer
4
5 # Create sample data
6 df = pd.DataFrame({
7     'age': [25, 30, None, 35, 40],
8     'income': [50000, 60000, 55000, None, 80000],
9     'score': [85, None, 90, 88, 92]
10 })
11
12 # Apply KNN Imputer
13 imputer = KNNImputer(n_neighbors=3)
14 df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
15
16 print("Original:")
17 print(df)
18
19 print("\nAfter KNN Imputation:")
20 print(df_imputed)
```

Listing 2.15: KNN Imputation using Scikit-learn

2.6 Summary: When to Use Which Imputation Method

Method	Best For	When to Use	Example
Drop rows	MCAR pattern	≤5% missing, random pattern	Random missing values
Drop columns	Low importance column	≤50% missing	“notes” column mostly empty
Mean imputation	Symmetric numeric data	No outliers, normal distribution	Test scores
Median imputation	Skewed numeric data	Has outliers	Income, house prices
Mode imputation	Categorical data	Nominal categories	City, gender
Forward/Backward fill	Time series	Consecutive missing	Stock prices, sensor data
Interpolation	Time series with trend	Gradual changes	Temperature readings
KNN imputation	Complex relationships	Many correlated features	Customer profiles
Constant imputation	Marking missing	Need to preserve missing flag	“Unknown” category

Handling Duplicates

What are Duplicates?

Duplicates are identical or nearly identical records that appear multiple times in a dataset.

Type	Description	Example
Exact Duplicates	All columns have identical values	Same customer row appears twice
Partial Duplicates	Only key columns match	Same email but different name
Near Duplicates	Values are similar but not identical	“John Smith” vs “Jon Smith”

Detecting Duplicates (Using Pandas)

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Check for duplicate rows
6 duplicate_rows = df.duplicated()
7 print(duplicate_rows.sum())
8
9 # View duplicate rows
10 duplicates = df[duplicate_rows]
11 print(duplicates)
12
13 # Check duplicates based on specific columns
14 duplicate_by_name = df.duplicated(subset=['name'])
15 duplicate_by_name_city = df.duplicated(subset=['name', 'city'])
16
17 # Keep first occurrence
18 df[df.duplicated(keep='first')]
19
20 # Keep last occurrence
21 df[df.duplicated(keep='last')]
22
23 # Mark duplicates
24 df['is_duplicate'] = df.duplicated()
```

Detecting Duplicates (Using CSV Module)

```
1 import csv
2
3 def detect_duplicates_csv(filename, key_columns=None):
4     with open(filename, 'r') as file:
5         reader = csv.DictReader(file)
6         rows = list(reader)
7
8         seen = {}
9         duplicates = []
10
11         if key_columns:
12             for i, row in enumerate(rows):
13                 key = tuple(row[col] for col in key_columns)
14                 if key in seen:
15                     duplicates.append((i, row))
16                 else:
17                     seen[key] = i
18         else:
19             for i, row in enumerate(rows):
20                 key = tuple(row.items())
21                 if key in seen:
22                     duplicates.append((i, row))
23                 else:
24                     seen[key] = i
25
26         print(f"Total rows: {len(rows)}")
27         print(f"Unique rows: {len(seen)}")
28         print(f"Duplicate rows: {len(duplicates)}")
29
30     return duplicates
```

3.3 Removing Duplicates

Using Pandas

```
1 import pandas as pd
2
3 df = pd.read_csv('customers.csv')
4
5 # Remove exact duplicates (keep first occurrence)
6 df_unique = df.drop_duplicates()
7 print(len(df), len(df_unique))
8
9 # Remove duplicates keeping last occurrence
10 df_unique = df.drop_duplicates(keep='last')
11
12 # Remove duplicates based on specific columns
13 df_unique = df.drop_duplicates(subset=['name', 'city'])
14
15 # Remove duplicates and reset index
16 df_unique = df.drop_duplicates().reset_index(drop=True)
17
18 # Count removed duplicates
```

```
19 before = len(df)
20 df_unique = df.drop_duplicates()
21 after = len(df_unique)
22 print(f"Removed {before - after} duplicate rows")
```

Using CSV Module

```
1 import csv
2
3 def remove_duplicates_csv(input_file, output_file, key_columns=None):
4     with open(input_file, 'r') as infile:
5         reader = csv.DictReader(infile)
6         rows = list(reader)
7
8         seen = set()
9         unique_rows = []
10
11     for row in rows:
12         if key_columns:
13             key = tuple(row[col] for col in key_columns)
14         else:
15             key = tuple(row.items())
16
17         if key not in seen:
18             seen.add(key)
19             unique_rows.append(row)
20
21     with open(output_file, 'w', newline='') as outfile:
22         writer = csv.DictWriter(outfile, fieldnames=reader.fieldnames)
23         writer.writeheader()
24         writer.writerows(unique_rows)
25
26     print(f"Original rows: {len(rows)}")
27     print(f"Unique rows: {len(unique_rows)}")
28     print(f"Removed duplicates: {len(rows) - len(unique_rows)}")
29
30     return unique_rows
31
32 # Usage
33 remove_duplicates_csv('customers.csv', 'customers_no_duplicates.csv', ['customer_id'])
34 remove_duplicates_csv('customers.csv', 'customers_no_duplicates_name.csv', ['name', 'email'])
```

3.4 Handling Near-Duplicates (Fuzzy Matching)

```
1 import pandas as pd
2 from difflib import SequenceMatcher
3
4 def similarity_ratio(a, b):
5     return SequenceMatcher(None, a.lower(), b.lower()).ratio()
6
7 def detect_near_duplicates(df, column, threshold=0.85):
8     duplicates = []
9     values = df[column].dropna().tolist()
10
11     for i in range(len(values)):
12         for j in range(i+1, len(values)):
13             ratio = similarity_ratio(values[i], values[j])
14
15             if ratio >= threshold and values[i] != values[j]:
16                 duplicates.append({
17                     'row1_index': i,
18                     'row1_value': values[i],
19                     'row2_index': j,
20                     'row2_value': values[j],
21                     'similarity': ratio
22                 })
23
24     return pd.DataFrame(duplicates)
25
26 data = pd.DataFrame({
27     'name': ['John Smith', 'Jon Smith', 'John Smyth', 'Alice Brown', 'Ali ce Brown']
28 })
29
30 near_dupes = detect_near_duplicates(data, 'name', threshold=0.8)
31 print(near_dupes)
```

3.5 Handling Duplicates with Aggregation

```
1 import pandas as pd
2
3 df = pd.DataFrame({
```

```
4     'customer_id': [1, 1, 2, 2, 3],
5     'name': ['Alice', 'Alice', 'Bob', 'Bob', 'Charlie'],
6     'amount': [100, 150, 200, 50, 300],
7     'date': ['2024-01-01', '2024-01-15', '2024-01-05', '2024-01-10', '2024-01-01']
8 })
9
10 # Aggregate duplicates
11 df_agg = df.groupby('customer_id').agg({
12     'name': 'first',
13     'amount': 'sum',
14     'date': 'min'
15 }).reset_index()
16
17 print(df_agg)
```

4. Complete Data Cleaning Pipeline Example

Example: Cleaning Customer Dataset

```
1 import pandas as pd
2 import numpy as np
3
4 print("=" * 70)
5 print("COMPLETE DATA CLEANING PIPELINE")
6
7 # ===== STEP 1: Load Data =====
8 df = pd.read_csv('customers_raw.csv')
9 print(df.shape)
10 print(list(df.columns))
11
12 # ===== STEP 2: Initial Inspection =====
13 print(df.isnull().sum())
14 print(df.duplicated().sum())
15
16 # ===== STEP 3: Remove Duplicates =====
17 initial_rows = len(df)
18 df = df.drop_duplicates()
19 print(initial_rows - len(df))
20
21 # ===== STEP 4: Handle Missing Values =====
22 missing_pct = (df.isnull().sum() / len(df)) * 100
23
24 # Drop high missing columns
25 high_missing_cols = missing_pct[missing_pct > 50].index
26 df = df.drop(columns=high_missing_cols)
27
28 # Numeric columns median imputation
29 numeric_cols = df.select_dtypes(include=[np.number]).columns
30 for col in numeric_cols:
31     if df[col].isnull().sum() > 0:
32         df[col].fillna(df[col].median(), inplace=True)
33
34 # Categorical columns mode imputation
35 categorical_cols = df.select_dtypes(include=['object']).columns
36 for col in categorical_cols:
37     if df[col].isnull().sum() > 0:
38         df[col].fillna(df[col].mode()[0], inplace=True)
39
40 # ===== STEP 5: Final Check =====
41 print(df.shape)
42 print(df.isnull().sum().sum())
43
44 # ===== STEP 6: Save Cleaned Data =====
45 df.to_csv('customers_cleaned.csv', index=False)
46 print("Saved cleaned dataset")
```

Data Integration and Transformation

1. What is Data Integration

Data Integration is the process of combining data from different sources (files, databases, APIs, spreadsheets) into a unified and consistent dataset.

1.1 Why Data Integration is Needed

Reason	Explanation	Example
Data silos	Data stored in separate systems	CRM vs ERP systems
Different formats	Multiple formats exist	CSV, Excel, JSON
Inconsistent IDs	Same entity has different identifiers	Customer ID mismatch
Complementary data	Each source has partial information	Demographics + transactions

2. Data Integration Operation: Concatenation

Concatenation refers to stacking datasets vertically by adding rows. All datasets must have the same columns.

Example: Pandas Concatenation

```
1 import pandas as pd
2
3 df_jan = pd.DataFrame({
4     'customer_id': [1, 2, 3],
5     'name': ['Alice', 'Bob', 'Charlie'],
6     'amount': [250, 180, 320]
7 })
8
9 df_feb = pd.DataFrame({
10     'customer_id': [4, 5, 6],
11     'name': ['Diana', 'Eve', 'Frank'],
12     'amount': [410, 280, 350]
13 })
14
15 print(df_jan)
16 print(df_feb)
17
18 df_all = pd.concat([df_jan, df_feb], axis=0)
19 print(df_all)
```

Output

January DataFrame:

	customer_id	name	amount
0	1	Alice	250
1	2	Bob	180
2	3	Charlie	320

February DataFrame:

	customer_id	name	amount
0	4	Diana	410
1	5	Eve	280
2	6	Frank	350

Concatenated Result (without resetting index):

	customer_id	name	amount
0	1	Alice	250
1	2	Bob	180
2	3	Charlie	320
0	4	Diana	410
1	5	Eve	280
2	6	Frank	350

Concatenation with Reset Index

```
1 df_all_reset = pd.concat([df_jan, df_feb], axis=0, ignore_index=True)
2 print(df_all_reset)
```

Output

	customer_id	name	amount
0	1	Alice	250
1	2	Bob	180
2	3	Charlie	320
3	4	Diana	410
4	5	Eve	280
5	6	Frank	350

Concatenation with Reset Index

```
1 df_all_reset = pd.concat([df_jan, df_feb], axis=0, ignore_index=True)
2 print(df_all_reset)
```

Output

Concatenated Result (with reset index):

	customer_id	name	amount
0	1	Alice	250
1	2	Bob	180
2	3	Charlie	320
3	4	Diana	410
4	5	Eve	280
5	6	Frank	350

Concatenating Multiple Files

```
1 import pandas as pd
2
3 df_jan = pd.DataFrame({'id': [1,2], 'sales': [100,200]})
4 df_feb = pd.DataFrame({'id': [3,4], 'sales': [300,400]})
5 df_mar = pd.DataFrame({'id': [5,6], 'sales': [500,600]})
6
7 df_q1 = pd.concat([df_jan, df_feb, df_mar], ignore_index=True)
8 print(df_q1)
9 print(df_q1.shape)
```

Output

January data:

	id	sales
0	1	100
1	2	200

February data:

	id	sales
0	3	300
1	4	400

March data:

	id	sales
0	5	500
1	6	600

Q1 Combined Data:

	id	sales
0	1	100
1	2	200
2	3	300
3	4	400
4	5	500
5	6	600

Q1 data shape: (6, 2)

Handling Different Columns During Concatenation

```
1 df1 = pd.DataFrame({'id': [1, 2], 'name': ['Alice', 'Bob']})
2 df2 = pd.DataFrame({'id': [3, 4], 'age': [25, 30]})
3
4 df_outer = pd.concat([df1, df2], axis=0, ignore_index=True)
5 print(df_outer)
6
7 df_inner = pd.concat([df1, df2], axis=0, ignore_index=True, join='inner')
8 print(df_inner)
```

Output

Outer Concatenation (fills missing with NaN):

	id	name	age
0	1	Alice	NaN
1	2	Bob	NaN
2	3	NaN	25.0
3	4	NaN	30.0

Inner Concatenation (only common columns):

	id
0	1
1	2
2	3
3	4

2.0.1 Concatenating CSV Files Using Python (CSV Module)

```
1 import csv
2
3 # Sample CSV files creation
4 with open('sales_jan.csv', 'w') as f:
5     f.write("id,product,sales\n1,A,100\n2,B,150")
6
7 with open('sales_feb.csv', 'w') as f:
8     f.write("id,product,sales\n3,C,200\n4,D,250")
9
```



```
10 print("Sample files created successfully")
11
12 def concatenate_csv_files(input_files, output_file):
13     first_file = True
14
15     for file in input_files:
16         with open(file, 'r') as infile:
17             reader = csv.reader(infile)
18
19             with open(output_file, 'a', newline='') as outfile:
20                 writer = csv.writer(outfile)
21
22                 for i, row in enumerate(reader):
23                     if i == 0 and first_file:
24                         writer.writerow(row)    # write header once
25                     elif i > 0:
26                         writer.writerow(row)
27
28     first_file = False
29     print(f"Processed: {file}")
30
31     print(f"\nFinal file saved as: {output_file}")
32
33     with open(output_file, 'r') as f:
34         print(f.read())
35
36 concatenate_csv_files(['sales_jan.csv', 'sales_feb.csv'], 'sales_q1.csv')
```

Sample files created successfully
Processed: sales_jan.csv
Processed: sales_feb.csv

Final file saved as: sales_q1.csv

id,product,sales
1,A,100
2,B,150
3,C,200
4,D,250

2.0.2 Merging (Joining) - Adding Columns Horizontally

Definition: Merging (or joining) combines datasets horizontally by using a common key column.

Sample Data for Join Operations

```
1 import pandas as pd
2
3 # Customers table
4 customers = pd.DataFrame({
5     'customer_id': [1, 2, 3, 4],
6     'name': ['Alice', 'Bob', 'Charlie', 'Diana'],
7     'city': ['New York', 'Los Angeles', 'Chicago', 'Houston']
8 })
9
10 # Orders table
11 orders = pd.DataFrame({
12     'order_id': [101, 102, 103, 104, 105],
13     'customer_id': [1, 2, 1, 3, 5],
14     'amount': [250, 180, 320, 410, 280],
15     'date': ['2024-01-15', '2024-01-20', '2024-02-01', '2024-02-10', '2024-02-15']
16 })
17
18 print("Customers Table:")
19 print(customers)
20
21 print("\nOrders Table:")
22 print(orders)
```

Customers Table:

	customer_id	name	city
0	1	Alice	New York
1	2	Bob	Los Angeles
2	3	Charlie	Chicago
3	4	Diana	Houston

Orders Table:

	order_id	customer_id	amount	date
0	101	1	250	2024-01-15
1	102	2	180	2024-01-20
2	103	1	320	2024-02-01
3	104	3	410	2024-02-10
4	105	5	280	2024-02-15

A. Inner Join (Keep Only Matching Keys)

```
1 inner_join = pd.merge(customers, orders, on='customer_id', how='inner')
2 print("INNER JOIN (customers with orders):")
3 print(inner_join)
```

INNER JOIN (customers with orders):

	customer_id	name	city	order_id	amount	date
0	1	Alice	New York	101	250	2024-01-15
1	1	Alice	New York	103	320	2024-02-01
2	2	Bob	Los Angeles	102	180	2024-01-20
3	3	Charlie	Chicago	104	410	2024-02-10

B. Left Join (Keep All Customers)

```
1 left_join = pd.merge(customers, orders, on='customer_id', how='left')
2 print("LEFT JOIN (all customers):")
3 print(left_join)
```

LEFT JOIN (all customers):

	customer_id	name	city	order_id	amount	date
0	1	Alice	New York	101.0	250.0	2024-01-15
1	1	Alice	New York	103.0	320.0	2024-02-01
2	2	Bob	Los Angeles	102.0	180.0	2024-01-20
3	3	Charlie	Chicago	104.0	410.0	2024-02-10
4	4	Diana	Houston	NaN	NaN	NaN

C. Right Join (Keep All Orders)

```
1 right_join = pd.merge(customers, orders, on='customer_id', how='right')
2 print("RIGHT JOIN (all orders):")
3 print(right_join)
```

RIGHT JOIN (all orders):

	customer_id	name	city	order_id	amount	date
0	1.0	Alice	New York	101	250	2024-01-15
1	1.0	Alice	New York	103	320	2024-02-01
2	2.0	Bob	Los Angeles	102	180	2024-01-20
3	3.0	Charlie	Chicago	104	410	2024-02-10
4	5.0	NaN	NaN	105	280	2024-02-15

D. Outer Join (Keep All Records)

```
1 outer_join = pd.merge(customers, orders, on='customer_id', how='outer')
2 print("OUTER JOIN (all customers + all orders):")
3 print(outer_join)
```

OUTER JOIN (all customers + all orders):

	customer_id	name	city	order_id	amount	date
0	1.0	Alice	New York	101.0	250.0	2024-01-15
1	1.0	Alice	New York	103.0	320.0	2024-02-01
2	2.0	Bob	Los Angeles	102.0	180.0	2024-01-20
3	3.0	Charlie	Chicago	104.0	410.0	2024-02-10
4	4.0	Diana	Houston	NaN	NaN	NaN
5	5.0	NaN	NaN	105.0	280.0	2024-02-15

2.0.3 Data Transformation

Definition: Data transformation is the process of converting data into a different format, scale, or structure to improve analysis and machine learning performance.

3.1 Scaling and Normalization

Sample Data

```
1 import pandas as pd
2 import numpy as np
3
4 df = pd.DataFrame({
5     'age': [25, 30, 35, 40, 45, 100],
6     'income': [30000, 45000, 60000, 75000, 90000, 120000],
7     'score': [85, 78, 92, 88, 95, 70]
8 })
9
10 print(df)
```

	age	income	score
0	25	30000	85
1	30	45000	78
2	35	60000	92
3	40	75000	88
4	45	90000	95
5	100	120000	70

```
1 df['age_minmax'] = (df['age'] - df['age'].min()) / (df['age'].max() - df['age'].min())
2 df['income_minmax'] = (df['income'] - df['income'].min()) / (df['income'].max() - df['income'].min())
3
4 print(df[['age', 'age_minmax', 'income', 'income_minmax']])
```

	age	age_minmax	income	income_minmax
0	25	0.000000	30000	0.000000
1	30	0.066667	45000	0.166667
2	35	0.133333	60000	0.333333
3	40	0.200000	75000	0.500000
4	45	0.266667	90000	0.666667
5	100	1.000000	120000	1.000000

```
1 df['age_zscore'] = (df['age'] - df['age'].mean()) / df['age'].std()
2 df['income_zscore'] = (df['income'] - df['income'].mean()) / df['income'].std()
3
4 print(df[['age', 'age_zscore', 'income', 'income_zscore']])
```

	age	age_zscore	income	income_zscore
0	25	-0.822222	30000	-1.336306
1	30	-0.622222	45000	-0.801784
2	35	-0.422222	60000	-0.267261
3	40	-0.222222	75000	0.267261
4	45	-0.022222	90000	0.801784
5	100	2.111111	120000	1.336306

Label Encoding

```
1 from sklearn.preprocessing import LabelEncoder
2
3 df = pd.DataFrame({
4     'city': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Houston']
5 })
6
7 le = LabelEncoder()
8 df['city_encoded'] = le.fit_transform(df['city'])
9
10 print(df)
```

	city	city_encoded
0	New York	3
1	Los Angeles	2
2	Chicago	0
3	New York	3
4	Houston	1

```
1 df_onehot = pd.get_dummies(df, columns=['city'])
2 print(df_onehot)
```

	city_Chicago	city_Houston	city_Los Angeles	city_New York
0	0	0	0	1
1	0	0	1	0
2	1	0	0	0
3	0	0	0	1
4	0	1	0	0

```
1 from sklearn.preprocessing import OrdinalEncoder
2
3 df = pd.DataFrame({
4     'education': ['High School', 'Bachelor', 'Master', 'PhD']
5 })
6
7 encoder = OrdinalEncoder(categories=[['High School', 'Bachelor', 'Master', 'PhD']])
8 df['education_encoded'] = encoder.fit_transform(df[['education']])
9
10 print(df)
```

	education	education_encoded
0	High School	0.0
1	Bachelor	1.0
2	Master	2.0
3	PhD	3.0

2.1 Handling Skewed Distributions

2.1.1 Example Dataset Creation

```
1 import pandas as pd
2 import numpy as np
3 from scipy import stats
4
5 np.random.seed(42)
6
7 df_skewed = pd.DataFrame({
8     'income': np.random.exponential(scale=50000, size=1000),
9     'price': np.random.lognormal(mean=3, sigma=1, size=1000)
10 })
11
12 print("Original Income Statistics:")
13 print("Skewness:", df_skewed['income'].skew())
14 print("Min:", df_skewed['income'].min())
15 print("Mean:", df_skewed['income'].mean())
16 print("Median:", df_skewed['income'].median())
17 print("Max:", df_skewed['income'].max())
```

Original Income Statistics:
Skewness: 2.01
Min: 136.81
Mean: 49914.97
Median: 34754.78
Max: 363653.77

2.1.2 Log Transformation

```
1 df_skewed['income_log'] = np.log1p(df_skewed['income'])
2 df_skewed['price_sqrt'] = np.sqrt(df_skewed['price'])
3
4 print("After Log Transformation:")
5 print("Skewness:", df_skewed['income_log'].skew())
6 print("Mean:", df_skewed['income_log'].mean())
7 print("Median:", df_skewed['income_log'].median())
```

After Log Transformation:
Skewness: 0.68
Mean: 10.41
Median: 10.46

2.1.3 Box-Cox Transformation

```
1 df_skewed['income_boxcox'], lambda_val = stats.boxcox(df_skewed['income'] + 1)
2
3 print("Box-Cox Transformation:")
4 print("Lambda:", lambda_val)
5 print("Skewness:", pd.Series(df_skewed['income_boxcox']).skew())
```

Box-Cox Transformation:
Lambda: 0.1765
Skewness: 0.03

2.2 Discretization (Binning)

2.2.1 Sample Data

```
1 import pandas as pd
2
3 df_bins = pd.DataFrame({
4     'age': [5, 12, 18, 25, 35, 45, 55, 65, 75, 85],
5     'income': [0, 5000, 15000, 30000, 50000,
6               75000, 100000, 150000, 200000, 250000]
7 })
8
9 print(df_bins)
```

	age	income
0	5	0
1	12	5000
2	18	15000
3	25	30000
4	35	50000
5	45	75000
6	55	100000
7	65	150000
8	75	200000
9	85	250000

2.2.2 Equal Width Binning

```
1 df_bins['age_bin'] = pd.cut(
2     df_bins['age'],
3     bins=4,
4     labels=['0-21', '21-42', '42-63', '63-84']
5 )
6
7 print(df_bins[['age', 'age_bin']])
```

	age	age_bin
0	5	0-21
1	12	0-21
2	18	0-21
3	25	21-42
4	35	21-42
5	45	42-63
6	55	42-63
7	65	63-84
8	75	63-84
9	85	63-84

2.2.3 Quantile Binning

```
1 df_bins['income_bin'] = pd.qcut(
2     df_bins['income'],
3     q=4,
4     labels=['Q1', 'Q2', 'Q3', 'Q4']
5 )
6
7 print(df_bins[['income', 'income_bin']])
```

	income	income_bin
0	0	Q1
1	5000	Q1
2	15000	Q1
3	30000	Q2
4	50000	Q2
5	75000	Q3
6	100000	Q3
7	150000	Q4
8	200000	Q4
9	250000	Q4

2.2.4 Custom Binning

```
1 age_bins = [0, 12, 19, 35, 60, 100]
2 labels = ['Child', 'Teen', 'Young Adult', 'Adult', 'Senior']
3
4 df_bins['age_group'] = pd.cut(
5     df_bins['age'],
6     bins=age_bins,
7     labels=labels
8 )
9
10 print(df_bins[['age', 'age_group']])
```

	age	age_group
0	5	Child
1	12	Child
2	18	Teen
3	25	Young Adult
4	35	Young Adult
5	45	Adult
6	55	Adult
7	65	Senior
8	75	Senior
9	85	Senior

2.2.5 Data Type Conversion

```
1 import pandas as pd
2
3 df_types = pd.DataFrame({
4     'id': ['1', '2', '3'],
5     'price': ['$49.99', '$29.99', '$99.99'],
6     'date': ['2024-01-15', '2024-02-20', '2024-03-10'],
7     'is_active': ['True', 'False', 'True']
8 })
```

```
9
10 print("Original Data:")
11 print(df_types)
12
13 print("\nOriginal Data Types:")
14 print(df_types.dtypes)
15
16 # Convert string to integer
17 df_types['id'] = df_types['id'].astype(int)
18
19 # Convert price string to float (remove $ sign)
20 df_types['price'] = df_types['price'].str.replace('$', '').astype(float)
21
22 # Convert to datetime
23 df_types['date'] = pd.to_datetime(df_types['date'])
24
25 # Convert to boolean
26 df_types['is_active'] = df_types['is_active'].map({'True': True, 'False': False})
27
28 print("\nAfter Conversion:")
29 print(df_types)
30
31 print("\nNew Data Types:")
32 print(df_types.dtypes)
```

Original Data:

	id	price	date	is_active
0	1	\$49.99	2024-01-15	True
1	2	\$29.99	2024-02-20	False
2	3	\$99.99	2024-03-10	True

Original Data Types:

id	object
price	object
date	object
is_active	object
dtype:	object

After Conversion:

	id	price	date	is_active
0	1	49.99	2024-01-15	True
1	2	29.99	2024-02-20	False
2	3	99.99	2024-03-10	True

New Data Types:

id	int64
price	float64
date	datetime64[ns]
is_active	bool
dtype:	object

4. Complete Data Integration and Transformation Pipeline

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler, LabelEncoder
4
5 print("=" * 70)
6 print("COMPLETE DATA INTEGRATION AND TRANSFORMATION PIPELINE")
7 print("=" * 70)
8
9 # ===== STEP 1: Load Multiple Data Sources =====
10 print("\n[1] Loading data from multiple sources...")
11
12 customers = pd.DataFrame({
13     'cust_id': [1, 2, 3, 4, 5],
14     'name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve'],
15     'city': ['NY', 'LA', 'CHI', 'HOU', 'PHX'],
16     'age': [25, 30, 35, 28, 42]
17 })
18
19 orders = pd.DataFrame({
20     'order_id': [101, 102, 103, 104, 105],
21     'customer_id': [1, 2, 1, 3, 99],
22     'amount': [250, 180, 320, 410, 280],
23     'date': ['2024-01-15', '2024-01-20', '2024-02-01', '2024-02-10', '2024-02-15']
24 })
25
26 print("Customers Table:")
27 print(customers)
28 print(f"\nCustomers shape: {customers.shape}")
```

```

29
30 print("\nOrders Table:")
31 print(orders)
32 print(f"Orders shape: {orders.shape}")
33
34 # ===== STEP 2: Data Integration =====
35 print("\n[2] Integrating data (merging customers and orders)...")
36
37 df_integrated = pd.merge(customers, orders,
38                           left_on='cust_id',
39                           right_on='customer_id',
40                           how='left')
41
42 print(f"Integrated shape: {df_integrated.shape}")
43 print("Integrated data (first 8 rows):")
44 print(df_integrated.head(8))
45
46 # ===== STEP 3: Handle Missing Values =====
47 print("\n[3] Handling missing values from integration...")
48
49 print("Missing values before handling:")
50 print(df_integrated.isnull().sum())
51
52 df_integrated['amount'].fillna(0, inplace=True)
53 df_integrated['has_order'] = df_integrated['order_id'].notna().astype(int)
54
55 print("\nMissing values after handling:")
56 print(df_integrated.isnull().sum())
57
58 # ===== STEP 4: Feature Engineering =====
59 print("\n[4] Creating new features...")
60
61 df_integrated['age_group'] = pd.cut(
62     df_integrated['age'],
63     bins=[0, 18, 30, 50, 100],
64     labels=['Youth', 'Young Adult', 'Adult', 'Senior']
65 )
66
67 df_integrated['amount_category'] = pd.cut(
68     df_integrated['amount'],
69     bins=[-1, 100, 300, 1000],
70     labels=['Low', 'Medium', 'High']
71 )
72
73 print("New features added:")
74 print(df_integrated[['age', 'age_group', 'amount', 'amount_category']].head())
75
76 # ===== STEP 5: Encoding =====
77 print("\n[5] Encoding categorical variables...")
78
79 label_encoder = LabelEncoder()
80 df_integrated['city_encoded'] = label_encoder.fit_transform(df_integrated['city'])
81
82 print("City encoding mapping:")
83 for idx, city in enumerate(label_encoder.classes_):
84     print(f" {city} -> {idx}")
85
86 df_integrated = pd.get_dummies(df_integrated, columns=['age_group'], prefix='age')
87
88 print("\nAfter encoding - new columns:")
89 print([col for col in df_integrated.columns if 'age_' in col])
90
91 # ===== STEP 6: Scaling =====
92 print("\n[6] Scaling numeric features...")
93
94 numeric_cols = ['age', 'amount']
95
96 print(f"Before scaling - Age range: [{df_integrated['age'].min()}, {df_integrated['age'].max()}]")
97 print(f"Before scaling - Amount range: [{df_integrated['amount'].min()}, {df_integrated['amount'].max()}]")
98
99 scaler = StandardScaler()
100 scaled_values = scaler.fit_transform(df_integrated[numeric_cols])
101
102 df_integrated['age_scaled'] = scaled_values[:, 0]
103 df_integrated['amount_scaled'] = scaled_values[:, 1]
104
105 print("\nAfter scaling:")
106 print(f"Age scaled - range: [{df_integrated['age_scaled'].min():.2f}, {df_integrated['age_scaled'].max():.2f}]")
107 print(f"Amount scaled - range: [{df_integrated['amount_scaled'].min():.2f}, {df_integrated['amount_scaled'].max():.2f}]")
108
109 # ===== STEP 7: Final Dataset =====
110 print("\n[7] Final integrated and transformed dataset:")
111 print(f"Final shape: {df_integrated.shape}")
112

```



```
113 print("\nAll columns in final dataset:")
114 for i, col in enumerate(df_integrated.columns, 1):
115     print(f" {i}. {col} ({df_integrated[col].dtype})")
116
117 print("\nSample data (first 5 rows):")
118 print(df_integrated.head())
119
120 # ===== STEP 8: Save =====
121 print("\n[8] Saving final dataset...")
122 df_integrated.to_csv('integrated_transformed_data.csv', index=False)
123
124 print("Saved to 'integrated_transformed_data.csv'")
125
126 print("\n" + "=" * 70)
127 print("DATA INTEGRATION AND TRANSFORMATION COMPLETED SUCCESSFULLY!")
128 print("=" * 70)
```

=====

COMPLETE DATA INTEGRATION AND TRANSFORMATION PIPELINE

=====

[1] Loading data from multiple sources...

Customers Table:

	cust_id	name	city	age
0	1	Alice	NY	25
1	2	Bob	LA	30
2	3	Charlie	CHI	35
3	4	Diana	HOU	28
4	5	Eve	PHX	42

Customers shape: (5, 4)

Orders Table:

	order_id	customer_id	amount	date
0	101	1	250	2024-01-15
1	102	2	180	2024-01-20
2	103	1	320	2024-02-01
3	104	3	410	2024-02-10
4	105	99	280	2024-02-15

Orders shape: (5, 4)

[2] Integrating data (merging customers and orders)...

Integrated shape: (6, 8)

Integrated data (first 8 rows):

	cust_id	name	city	age	order_id	customer_id	amount	date
0	1	Alice	NY	25	101.0	1.0	250.0	2024-01-15
1	1	Alice	NY	25	103.0	1.0	320.0	2024-02-01
2	2	Bob	LA	30	102.0	2.0	180.0	2024-01-20
3	3	Charlie	CHI	35	104.0	3.0	410.0	2024-02-10
4	4	Diana	HOU	28	NaN	NaN	NaN	NaN
5	5	Eve	PHX	42	NaN	NaN	NaN	NaN

[3] Handling missing values from integration...

Missing values before handling:

cust_id	0
name	0
city	0
age	0
order_id	2
customer_id	2
amount	2
date	2

dtype: int64

Missing values after handling:

cust_id	0
name	0
city	0
age	0
order_id	2
customer_id	2
amount	0
date	2
has_order	0

dtype: int64

[4] Creating new features...

New features added:

	age	age_group	amount	amount_category
0	25	Young Adult	250.0	Medium

1	25	Young Adult	320.0	High
2	30	Young Adult	180.0	Medium
3	35	Adult	410.0	High
4	28	Young Adult	0.0	Low
5	42	Adult	0.0	Low

[5] Encoding categorical variables...

City encoding mapping:

CHI -> 0
HOU -> 1
LA -> 2
NY -> 3
PHX -> 4

After encoding - new columns:

['age_group_Adult', 'age_group_Senior', 'age_group_Young Adult', 'age_group_Youth']

[6] Scaling numeric features...

Before scaling - Age range: [25, 42]

Before scaling - Amount range: [0.0, 410.0]

After scaling:

Age scaled - range: [-1.41, 1.18]

Amount scaled - range: [-1.12, 1.55]

[7] Final integrated and transformed dataset:

Final shape: (6, 15)

All columns in final dataset:

- 1. cust_id (int64)
- 2. name (object)
- 3. city (object)
- 4. age (int64)
- 5. order_id (float64)
- 6. customer_id (float64)
- 7. amount (float64)
- 8. date (object)
- 9. has_order (int64)
- 10. amount_category (category)
- 11. city_encoded (int64)
- 12. age_group_Adult (bool)
- 13. age_group_Senior (bool)
- 14. age_group_Young Adult (bool)
- 15. age_group_Youth (bool)

Sample data (first 5 rows):

	cust_id	name	city	age	order_id	customer_id	amount	date	has_order	amount_category	city_encoded	age_group
0	1	Alice	NY	25	101.0	1.0	250.0	2024-01-15	1	Medium	3	Young Adult
1	1	Alice	NY	25	103.0	1.0	320.0	2024-02-01	1	High	3	Young Adult
2	2	Bob	LA	30	102.0	2.0	180.0	2024-01-20	1	Medium	2	Young Adult
3	3	Charlie	CHI	35	104.0	3.0	410.0	2024-02-10	1	High	0	Adult
4	4	Diana	HOU	28	NaN	NaN	0.0	NaN	0	Low	1	Adult

[8] Saving final dataset...

Saved to 'integrated_transformed_data.csv'

=====

DATA INTEGRATION AND TRANSFORMATION COMPLETED SUCCESSFULLY!

=====

5. Quick Reference Summary

Integration Operations

Operation	Pandas Code	Description
Concatenate rows	pd.concat([df1, df2], axis=0)	Stack vertically
Concatenate columns	pd.concat([df1, df2], axis=1)	Stack horizontally
Inner join	pd.merge(df1, df2, on='key', how='inner')	Keep only matches
Left join	pd.merge(df1, df2, on='key', how='left')	Keep all from left
Right join	pd.merge(df1, df2, on='key', how='right')	Keep all from right
Outer join	pd.merge(df1, df2, on='key', how='outer')	Keep all from both

Transformation Operations

Operation	Pandas Code	Output Range
Min-Max scaling	(x - min)/(max - min)	[0, 1]
Standardization	(x - mean)/std	mean=0, std=1
Label Encoding	LabelEncoder().fit_transform()	0 to n-1
One-Hot Encoding	pd.get_dummies()	0/1 columns
Log Transformation	np.log1p(x)	Reduces skewness

Feature Engineering and Feature Selection

1. What is Feature Engineering?

Feature Engineering is the process of creating new features from existing data to improve machine learning model performance.
Analogy: If raw data is flour, feature engineering is adding sugar, eggs, and butter to make a cake.

2. Types of Feature Engineering

2.1 Creating Features from Numeric Data

```
1 import pandas as pd
2 import numpy as np
3
4 df = pd.DataFrame({
5     'height': [160, 175, 168, 182],
6     'weight': [60, 75, 68, 85],
7     'price': [100, 200, 150, 300],
8     'qty': [5, 3, 8, 2]
9 })
10
11 print("Original Data:")
12 print(df)
13
14 # Ratio feature
15 df['bmi'] = df['weight'] / ((df['height']/100)**2)
16
17 # Product feature
18 df['total_price'] = df['price'] * df['qty']
19
20 print("\nAfter Feature Engineering:")
21 print(df)
```

Output:

Original Data:

	height	weight	price	qty
0	160	60	100	5
1	175	75	200	3
2	168	68	150	8
3	182	85	300	2

After Feature Engineering:

	height	weight	price	qty	bmi	total_price
0	160	60	100	5	23.437500	500
1	175	75	200	3	24.489796	600
2	168	68	150	8	24.092970	1200
3	182	85	300	2	25.662968	600

2.2 Creating Features from Date/Time

```
1 import pandas as pd
2
3 df = pd.DataFrame({'date': pd.date_range('2024-01-01', periods=5)})
4
5 print("Original Data:")
6 print(df)
7
8 # Extract date features
9 df['day'] = df['date'].dt.day
10 df['month'] = df['date'].dt.month
11 df['day_of_week'] = df['date'].dt.dayofweek # Monday=0
12 df['is_weekend'] = (df['day_of_week'] >= 5).astype(int)
13
14 print("\nAfter Date Features:")
15 print(df)
```

Output:

Original Data:

date

```
0 2024-01-01
1 2024-01-02
2 2024-01-03
3 2024-01-04
4 2024-01-05
```

After Date Features:

	date	day	month	day_of_week	is_weekend
0	2024-01-01	1	1	0	0
1	2024-01-02	2	1	1	0
2	2024-01-03	3	1	2	0
3	2024-01-04	4	1	3	0
4	2024-01-05	5	1	4	0

2.3 Creating Features from Text

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     'review': ['Great product!', 'Very bad quality', 'Good value for money']
5 })
6
7 print("Original Data:")
8 print(df)
9
10 # Text features
11 df['word_count'] = df['review'].str.split().str.len()
12 df['char_count'] = df['review'].str.len()
13 df['has_exclamation'] = df['review'].str.contains('!').astype(int)
14
15 print("\nAfter Text Features:")
16 print(df)
```

Output:

Original Data:

	review
0	Great product!
1	Very bad quality
2	Good value for money

After Text Features:

	review	word_count	char_count	has_exclamation
0	Great product!	2	14	1
1	Very bad quality	3	17	0
2	Good value for money	4	21	0

2.4 Creating Features from Aggregations (Group By)

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     'customer': ['A', 'A', 'B', 'B', 'B'],
5     'amount': [100, 200, 150, 300, 250]
6 })
7
8 print("Original Data:")
9 print(df)
10
11 # Aggregate features per customer
12 agg = df.groupby('customer')['amount'].agg(['sum', 'mean', 'count']).reset_index()
13 agg.columns = ['customer', 'total_amount', 'avg_amount', 'transaction_count']
14
15 print("\nAfter Aggregation Features:")
16 print(agg)
```

Output:

Original Data:

	customer	amount
0	A	100
1	A	200
2	B	150
3	B	300
4	B	250

After Aggregation Features:

	customer	total_amount	avg_amount	transaction_count
0	A	300	100.0	2
1	B	700	233.3	3

3. What is Feature Selection?

Feature Selection is the process of choosing the most important features and removing irrelevant or redundant ones.

Benefits:

- Reduces overfitting
- Improves accuracy
- Reduces training time
- Makes model simpler to understand

4. Types of Feature Selection

4.1 Filter Methods (Statistical Tests)

```
1 import pandas as pd
2 from sklearn.datasets import make_classification
3 from sklearn.feature_selection import SelectKBest, f_classif
4
5 # Create sample data
6 X, y = make_classification(n_samples=100, n_features=5, random_state=42)
7 df = pd.DataFrame(X, columns=['f1', 'f2', 'f3', 'f4', 'f5'])
8
9 print("Feature Scores (ANOVA F-test):")
10 selector = SelectKBest(score_func=f_classif, k=3)
11 selector.fit(X, y)
12
13 for i, score in enumerate(selector.scores_):
14     print(f"Feature f{i+1}: {score:.2f}")
```

Output:

```
Feature Scores (ANOVA F-test):
Feature f1: 4.82
Feature f2: 37.75
Feature f3: 0.19
Feature f4: 27.39
Feature f5: 4.21
```

4.2 Correlation Method

```
1 import pandas as pd
2
3 df = pd.DataFrame({
4     'age': [25, 30, 35, 40, 45],
5     'income': [30000, 45000, 60000, 75000, 90000],
6     'score': [85, 78, 92, 88, 95],
7     'target': [0, 0, 1, 1, 1]
8 })
9
10 print("Correlation with Target:")
11 print(df.corr()['target'].sort_values(ascending=False))
```

Output:

```
Correlation with Target:
target    1.000000
income    0.948683
age       0.948683
score     0.258199
Name: target, dtype: float64
```

4.3 Variance Threshold (Remove Constant Features)

```
1 from sklearn.feature_selection import VarianceThreshold
2
3 df = pd.DataFrame({
4     'constant': [1, 1, 1, 1, 1],
5     'low_variance': [1, 1, 1, 2, 2],
6     'high_variance': [1, 5, 3, 8, 2]
7 })
8
9 print("Original Data:")
10 print(df)
11
12 selector = VarianceThreshold(threshold=0.5)
13 selected = selector.fit_transform(df)
14
15 print(f"\nRemoved columns with variance < 0.5")
16 print(f"Original shape: {df.shape}")
17 print(f"After selection shape: {selected.shape}")
```

Output:

Original Data:

	constant	low_variance	high_variance
0	1	1	1
1	1	1	5
2	1	1	3
3	1	2	8
4	1	2	2

Removed columns with variance < 0.5
Original shape: (5, 3)
After selection shape: (5, 2)

4.4 Wrapper Method (RFE - Recursive Feature Elimination)

```
1 from sklearn.feature_selection import RFE
2 from sklearn.linear_model import LogisticRegression
3 import pandas as pd
4
5 # Sample data
6 df = pd.DataFrame({
7     'f1': [1, 2, 3, 4, 5],
8     'f2': [5, 4, 3, 2, 1],
9     'f3': [2, 3, 2, 3, 2],
10    'f4': [1, 1, 2, 2, 3],
11    'target': [0, 0, 1, 1, 1]
12 })
13
14 X = df[['f1', 'f2', 'f3', 'f4']]
15 y = df['target']
16
17 # RFE
18 model = LogisticRegression()
19 rfe = RFE(model, n_features_to_select=2)
20 rfe.fit(X, y)
21
22 print("Feature Selection Results:")
23 for i, selected in enumerate(rfe.support_):
24     print(f"Feature f{i+1}: {'Selected' if selected else 'Removed'}")
```

Output:

Feature Selection Results:
Feature f1: Selected
Feature f2: Removed
Feature f3: Removed
Feature f4: Selected

4.5 Embedded Method (Feature Importance from Random Forest)

```
1 from sklearn.ensemble import RandomForestClassifier
2 import pandas as pd
3
4 df = pd.DataFrame({
5     'age': [25, 30, 35, 40, 45, 50, 55, 60],
6     'income': [30, 45, 60, 75, 90, 100, 80, 70],
7     'score': [85, 78, 92, 88, 95, 85, 90, 88],
8     'purchases': [2, 3, 5, 4, 6, 7, 5, 4],
9     'target': [0, 0, 1, 1, 1, 1, 0, 0]
10 })
11
12 X = df.drop('target', axis=1)
13 y = df['target']
14
15 model = RandomForestClassifier()
16 model.fit(X, y)
17
18 importance = pd.DataFrame({
19     'feature': X.columns,
20     'importance': model.feature_importances_
21 }).sort_values('importance', ascending=False)
22
23 print("Feature Importance:")
24 print(importance)
```

Output:

Feature Importance:

	feature	importance
2	score	0.456789
1	income	0.321456
0	age	0.123456
3	purchases	0.098299

5. Quick Summary Table

Feature Engineering	Feature Selection
Create new features	Remove existing features
Increases features	Decreases features
Uses domain knowledge	Uses statistics/algorithms
Examples: BMI, total price	Examples: correlation, RFE

Method	Type	How it works
Correlation	Filter	Remove highly correlated features
Variance Threshold	Filter	Remove constant/low variance features
RFE	Wrapper	Recursively remove least important
Random Forest Importance	Embedded	Use model's built-in importance

2.3 Dimensionality Reduction

2.3.1 1. What is Dimensionality Reduction?

Dimensionality Reduction is the process of reducing the number of features by creating new features that are combinations of the original ones.

Analogy: Instead of keeping 10 different measurements of a fruit (weight, height, width, color intensity, etc.), you create 2 “super-features” that capture most of the information.

2.3.2 2. Why Do We Need Dimensionality Reduction?

Problem	Explanation
Curse of Dimensionality	As features increase, data becomes sparse and models fail
High Computational Cost	More features = slower training
Storage Space	Large datasets take more memory
Overfitting	Too many features = model memorizes noise
Visualization	Can't visualize > 3 dimensions

2.3.3 3. Feature Selection vs Dimensionality Reduction

```
1 import pandas as pd
2
3 # Original features
4 df = pd.DataFrame({
5     'height': [160, 170, 180],
6     'weight': [60, 70, 80],
7     'age': [25, 30, 35]
8 })
9
10 print("Original Features:", list(df.columns))
11
12 # Feature Selection
13 selected = df[['height', 'weight']]
14 print("Feature Selection - keeps existing features:", list(selected.columns))
15
16 # Dimensionality Reduction
17 print("Dimensionality Reduction - creates NEW features: PC1, PC2 (not original)")
```

Original Features: ['height', 'weight', 'age']
Feature Selection - keeps existing features: ['height', 'weight']
Dimensionality Reduction - creates NEW features: PC1, PC2 (not original)

2.3.4 4. Principal Component Analysis (PCA)

4.1 How PCA Works

2.4 PCA Algorithm – Step by Step

Step 1: Arrange the Data

Organize the data into a matrix X with m rows (samples) and n columns (features).

Step 2: Calculate the Mean of Each Feature

Compute the mean for every column (feature) in the dataset.

Mean = (Sum of all values in column) / (Number of rows)

Step 3: Center the Data

Subtract the mean of each column from every value in that column.

Centered Data = Original Data - Mean

Step 4: Calculate the Covariance Matrix

Compute the covariance matrix C using the formula:

$$C = (1 / (n-1)) \times (A \times A^T)$$

Where:

- A = Centered data matrix
- A^T = Transpose of A
- n = Number of samples

Step 5: Calculate Eigenvalues and Eigenvectors

Find the eigenvalues (λ) and eigenvectors (v) of the covariance matrix C by solving:

$$|C - \lambda I| = 0 \quad (\text{for eigenvalues})$$

$$(C - \lambda I) \times v = 0 \quad (\text{for eigenvectors})$$

Where I is the identity matrix.

Step 6: Sort Eigenvalues and Eigenvectors

Sort the eigenvalues in descending order (largest to smallest). Arrange the corresponding eigenvectors in the same order.

Step 7: Select Principal Components

Choose the top k eigenvectors (where k is the desired number of dimensions after reduction).

- $k = 1$ for 1D reduction
- $k = 2$ for 2D reduction
- $k = 3$ for 3D reduction (visualization)

Step 8: Form the Feature Vector (Transformation Matrix)

Create a matrix W by placing the selected eigenvectors as columns.

$$W = [v_1, v_2, v_3, \dots, v_k]$$

Step 9: Transform the Original Data

Multiply the centered data matrix A by the feature vector W to get the final reduced data.

$$\text{Final Data} = A \times W$$

Step 10: Result

The output is a new dataset with m rows (same number of samples) and k columns (reduced features).

2.5 PCA – Complete Numerical Example

Problem Statement

We have 3 features and 4 data points. Reduce the data from 3 dimensions to 2 dimensions using PCA.

Original Data

Data Point	Feature X_1	Feature X_2	Feature X_3
Point 1	1	2	4
Point 2	2	3	5
Point 3	3	4	6
Point 4	4	5	7

Step 1: Arrange Data in Matrix Form

```

1 X = [1  2  4]
2     [2  3  5]
3     [3  4  6]
4     [4  5  7]
```

Step 2: Calculate Mean of Each Feature

Feature	Calculation	Mean
X_1	$(1+2+3+4)/4$	2.5
X_2	$(2+3+4+5)/4$	3.5
X_3	$(4+5+6+7)/4$	5.5

```

1 M = [2.5, 3.5, 5.5]
```


Step 3: Center the Data

```
1 A = [-1.5  -1.5  -1.5]
2     [-0.5  -0.5  -0.5]
3     [ 0.5   0.5   0.5]
4     [ 1.5   1.5   1.5]
```

All columns are identical because features are perfectly correlated.

Step 4: Covariance Matrix

Step 4.1: Compute $A^T \times A$

```
1 A^T = [-1.5  -0.5   0.5   1.5]
2       [-1.5  -0.5   0.5   1.5]
3       [-1.5  -0.5   0.5   1.5]
```

Each element = 5.0

```
1 A^T A = [5.0  5.0  5.0]
2         [5.0  5.0  5.0]
3         [5.0  5.0  5.0]
```

Step 4.2: Divide by (n-1) = 3

```
1 C = [1.6667  1.6667  1.6667]
2      [1.6667  1.6667  1.6667]
3      [1.6667  1.6667  1.6667]
```

Step 5: Eigenvalues

1 = 5.0
2 = 0
3 = 0

Step 6: Eigenvectors

```
1 v1 = [0.57735, 0.57735, 0.57735]
2 v2 = [0.7071, -0.7071, 0]
3 v3 = [0.4082, 0.4082, -0.8165]
```

Step 7: Select Principal Components

Component	Eigenvalue	Variance
PC1	5.0	100%
PC2	0	0%
PC3	0	0%

Step 8: Feature Vector

```
1 W = [0.57735  0.7071 ]
2      [0.57735 -0.7071 ]
3      [0.57735  0      ]
```

Step 9: Transform Data

Point 1 -> (-2.598, 0)
Point 2 -> (-0.866, 0)
Point 3 -> (0.866, 0)
Point 4 -> (2.598, 0)

Step 10: Final Reduced Data

Point	PC1	PC2
(1,2,4)	-2.598	0
(2,3,5)	-0.866	0
(3,4,6)	0.866	0
(4,5,7)	2.598	0

Summary

```
1 Original:
2 [1  2  4]
3 [2  3  5]
4 [3  4  6]
5 [4  5  7]
6
7 Reduced:
8 [-2.598  0]
9 [-0.866  0]
10 [ 0.866  0]
11 [ 2.598  0]
```

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.decomposition import PCA
4
5 df = pd.DataFrame({
6     'height': [160, 165, 170, 175, 180, 185],
7     'weight': [55, 60, 65, 70, 75, 80],
8     'age': [25, 28, 30, 32, 35, 38]
9 })
10
11 print("Original Data (3 features):")
12 print(df)
13 print(f"Shape: {df.shape}")
14
15 pca = PCA(n_components=2)
16 pca_result = pca.fit_transform(df)
17
18 df_pca = pd.DataFrame(pca_result, columns=['PC1', 'PC2'])
19
20 print("\nAfter PCA (2 new features):")
21 print(df_pca)
22 print(f"Shape: {df_pca.shape}")
23
24 print("\nVariance explained by each component:")
25 print(f"PC1: {pca.explained_variance_ratio_[0]:.2%}")
26 print(f"PC2: {pca.explained_variance_ratio_[1]:.2%}")
27 print(f"Total: {sum(pca.explained_variance_ratio_):.2%}")
```

Original Data (3 features):

	height	weight	age
0	160	55	25
1	165	60	28
2	170	65	30
3	175	70	32
4	180	75	35
5	185	80	38

Shape: (6, 3)

After PCA (2 new features):

	PC1	PC2
0	-4.958595	0.219729
1	-2.989042	-0.560407
2	-0.970257	-0.309991
3	0.930767	-0.109352
4	2.932185	0.246868
5	5.054941	0.513152

Shape: (6, 2)

Variance explained by each component:
PC1: 99.74%
PC2: 0.26%
Total: 100.00%

4.2 Choosing Number of Components

```
1 from sklearn.decomposition import PCA
2
3 pca_full = PCA()
4 pca_full.fit(df)
5
6 cumulative_variance = np.cumsum(pca_full.explained_variance_ratio_)
7
8 print("Explained Variance Ratio:")
9 for i, var in enumerate(pca_full.explained_variance_ratio_, 1):
10     print(f"PC{i}: {var:.2%}")
11
12 print("\nCummulative explained variance:")
13 for i, cum in enumerate(cumulative_variance, 1):
14     print(f"First {i} components: {cum:.2%}")
```

Explained Variance Ratio:
PC1: 99.28%
PC2: 0.67%
PC3: 0.05%

Cumulative explained variance:
First 1 components: 99.28%
First 2 components: 99.95%
First 3 components: 100.00%

2.5.1 5. When to Use Feature Selection vs Dimensionality Reduction

Scenario	Feature Selection	PCA
Need interpretability	Yes	No
Highly correlated features	No	Yes
Visualization	No	Yes
Fast training	Yes	Yes
Noise reduction	No	Yes
Simple explanation	Yes	No

2.5.2 6. Complete Comparison Table

Aspect	Feature Selection	Dimensionality Reduction (PCA)
Output	Subset of original features	New features (Principal Components)
Interpretability	High	Low
Information loss	Possible	Minimizes loss
Correlation handling	Removes correlated features	Converts to uncorrelated components
Example	Keep height, weight	PC1 = combination of features

2.6 Practice Questions

SECTION A: 2-Mark Questions

1. What is data cleaning? List any two common data quality issues.
2. Define CSV file. What is the default delimiter used in CSV?
3. What is the difference between structured and unstructured data? Give one example of each.
4. Define API. What does REST stand for?
5. What is the difference between MCAR and MNAR?
6. Define feature engineering. Give one example.
7. What is the difference between feature selection and dimensionality reduction?
8. Define PCA in one sentence.
9. What is one-hot encoding? When is it used?
10. What is the difference between mean imputation and median imputation?

SECTION B: 4-Mark Questions

1. Explain the difference between CSV Reader and CSV DictReader with example use cases.
2. Explain any four methods for handling missing values with appropriate use cases.
3. Differentiate between inner join, left join, and outer join in data integration with examples.
4. Explain the four measurement scales of data (nominal, ordinal, interval, ratio) with examples.
5. Explain the difference between label encoding and one-hot encoding. When would you use each?
6. Explain any four date/time features extracted from timestamp data.
7. What is feature selection? Explain any two filter methods.
8. Explain the difference between min-max scaling and standardization (Z-score). When is each used?
9. What is PCA? List any three benefits of PCA.
10. Explain the difference between exact duplicates and near-duplicates and how to handle them.

SECTION C: 8-Mark Questions

1. Explain the complete data cleaning process including missing values (4 methods), duplicates, and inconsistent formats with examples.
2. Explain data integration including concatenation, joins (inner, left, right, outer), and conflict resolution strategies.
3. Explain data transformation techniques including scaling, encoding, and skewness handling with examples.
4. Explain feature engineering including numeric, date-time, and text-based feature creation with examples.
5. Explain feature selection methods: filter, wrapper, and embedded methods with advantages and disadvantages.
6. Explain PCA step by step with a complete numerical example including covariance matrix and eigen decomposition.
7. Compare feature selection and dimensionality reduction (PCA) with scenarios.
8. Explain preprocessing of dataset with columns: $customer_id$, $transaction_date$, $amount$, $product_category$, $city$, $review_text$.
9. Compare REST API and SOAP API on 8 parameters and provide Python code to fetch REST API data.
10. Explain curse of dimensionality and how PCA solves it, including variance explained concept.